

Chapter 4

Network Layer

A note on the

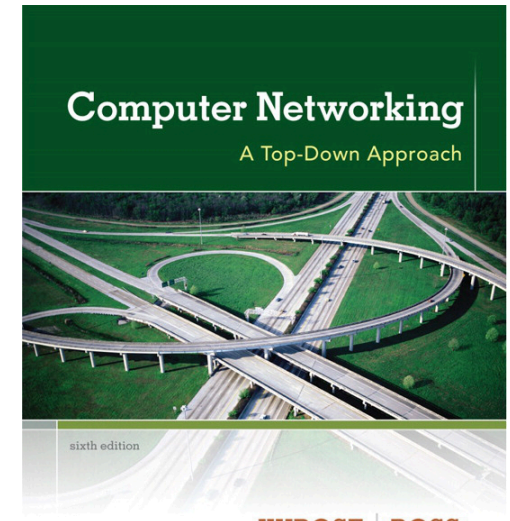
We're making these

They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2013
J.F Kurose and K.W. Ross, All Rights Reserved



*Networking: A
Top Down
Approach
6th edition
Jim Kurose, Keith
Ross
Addison-Wesley
March 2012*

Chapter 4: network layer

chapter goals:

- ❖ understand principles behind network layer service
 - how a router works
 - routing (path selection)
 - broadcast, multicast
- ❖ instantiation, implementation in the Internet

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and datagram netw

4.4 IP: I Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

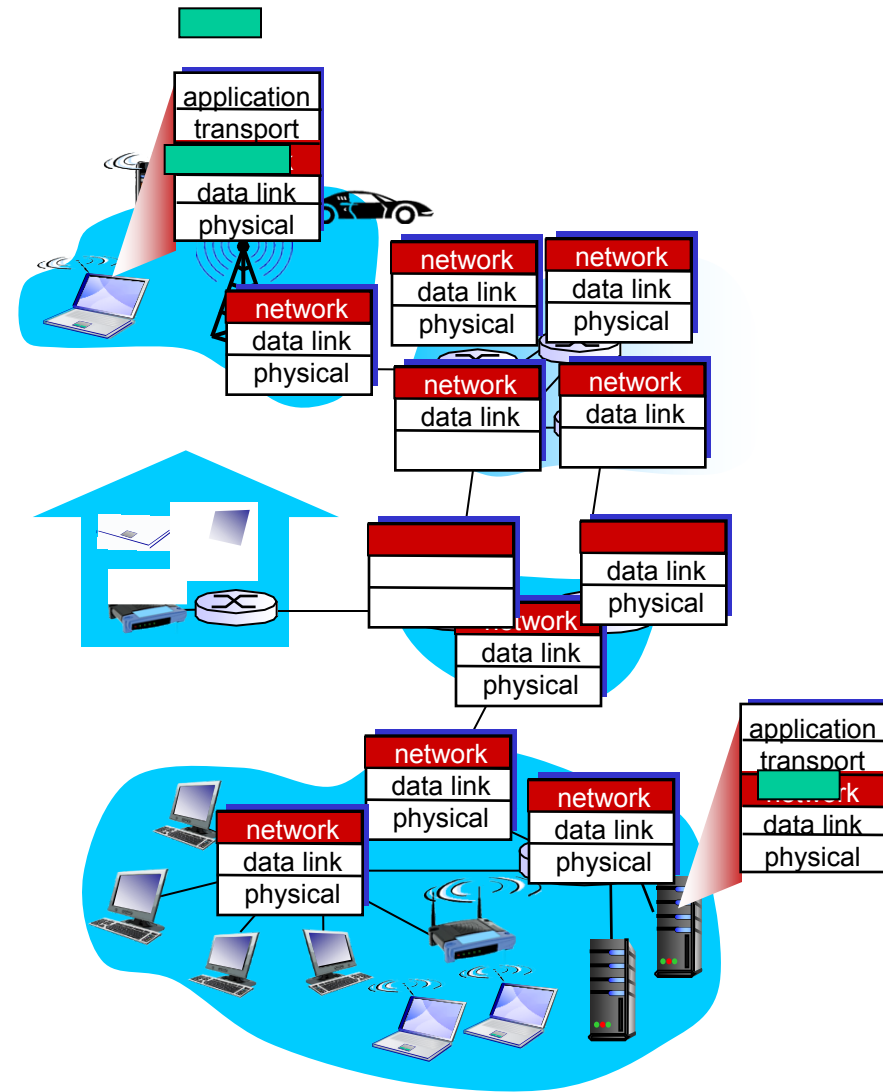
- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and multicast routing

Network layer

- ❖ transport segment from sending to receiving host
- ❖ on sending side
 - data
- ❖ on receiving side, delivers segments to transport layer
- ❖ network layer protocols in *every* host, router
- ❖ router examines header fields in all IP



Two key network-layer functions

❖ *forwarding*: move packets from router's input

analogy:

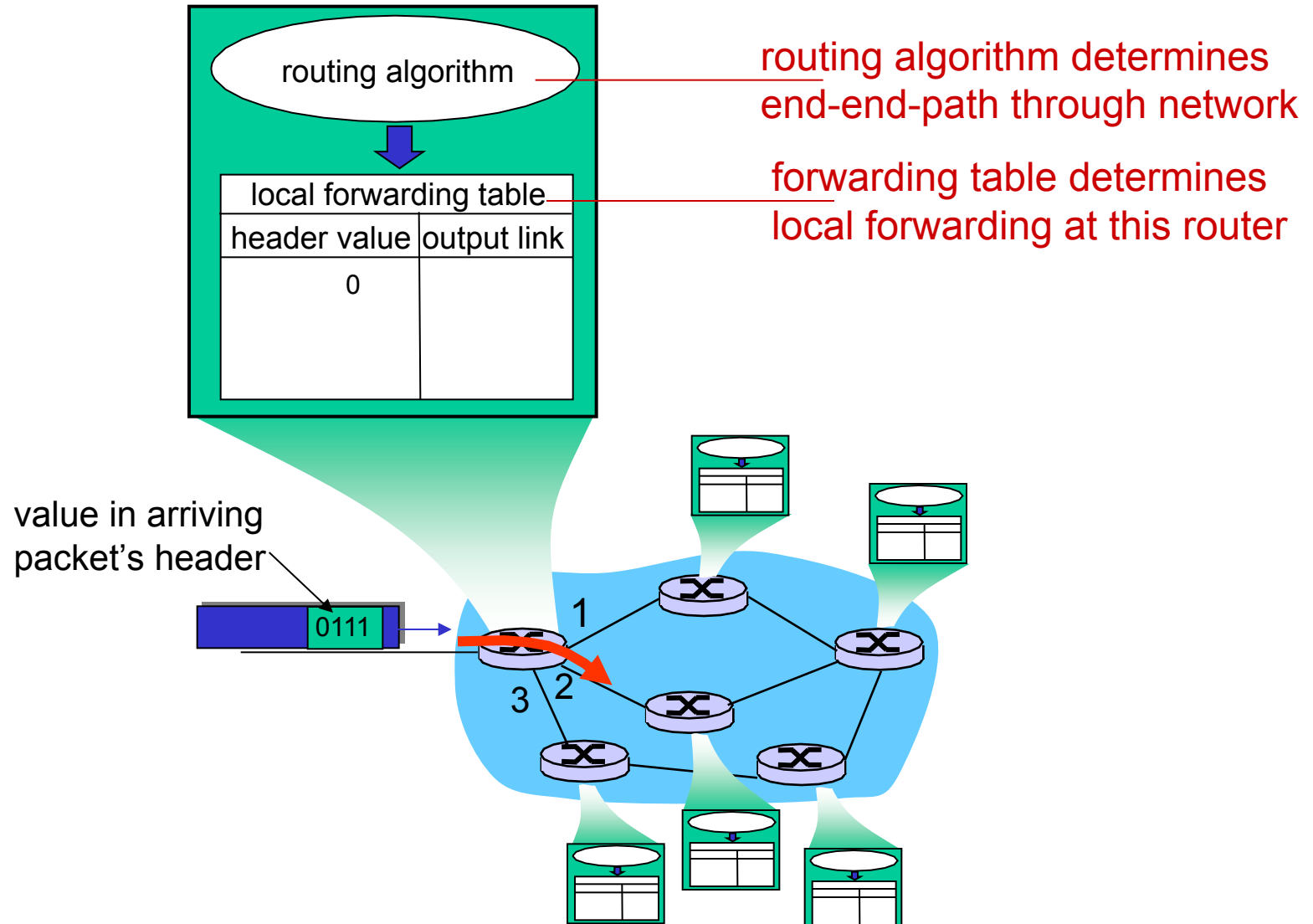
ting: process of

❖ *routing*: determine route taken by packets from source to dest.

❖ *forwarding*: process of getting through single interchange

- *routing algorithms*

Interplay between routing and forwarding



Connection setup

- ❖ 3rd important function in *some* network architectures:

- ATM, frame r

conn

- routers get involved
- ❖ network vs transport layer connection service:
 - *network*: between two hosts (may also involve intervening routers in case of VCs)
 - *transport*: between two processes

Network service model

Q: What *service model* for “channel”
transporting datagrams from sender to
receiver?
e *le services*

- ❖ guaranteed delivery
- ❖ guaranteed delivery with less than 40 msec delay
- ❖ in-order datagram delivery
- ❖ guaranteed minimum bandwidth to flow
- ❖ restrictions on changes in inter-packet spacing

Network layer service models:

Network Architecture	Service Model	Guarantees ?				Congestion feedback
		Bandwidth	Loss	Order	Timing	
Internet	best effort				no	no (inferred congestion)
ATM	VBR	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR	guaranteed minimum	no	yes	no	yes
ATM	UBR	none	no	yes	no	no

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and
datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and
multicast routing

Connection, connection-less ~~service~~

- ❖ *datagram* network provides network-layer *connectionless* service
- ❖ *virtual-circuit* provides
- ❖ orien - layer services, but:
 - *service*: host-to-host
 - *no choice*: network provides one or the other
 - *implementation*: in network core

Virtual circuits

“source-to-dest path behaves much like telephone circuit”

- performan

- ❖ call s data can flow
- ❖ each packet carries VC identifier (not destination host address)
- ❖ *every* router on source-dest path maintains “state” for each passing connection
- ❖ link, router resources (bandwidth, buffers) may be *allocated* to VC (dedicated resources = predictable service)

VC implementation

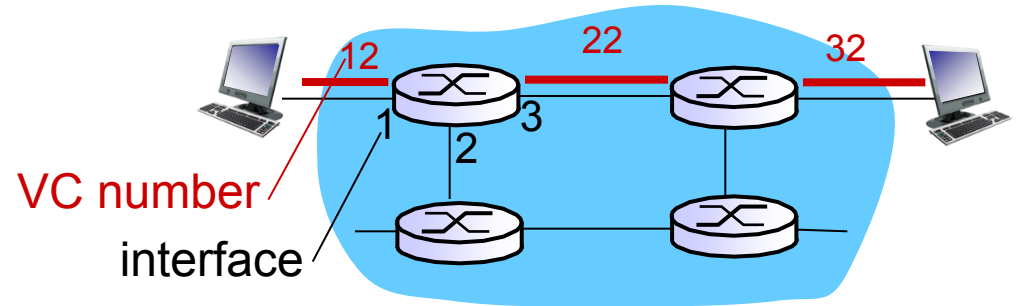
a VC consists of:

1. *path* from source to destination
2. *VC number* r each link

al

- ❖ packet belonging to VC carries VC number (rather than dest address)
- ❖ VC number can be changed on each link.
 - new VC number comes from forwarding table

VC forwarding table



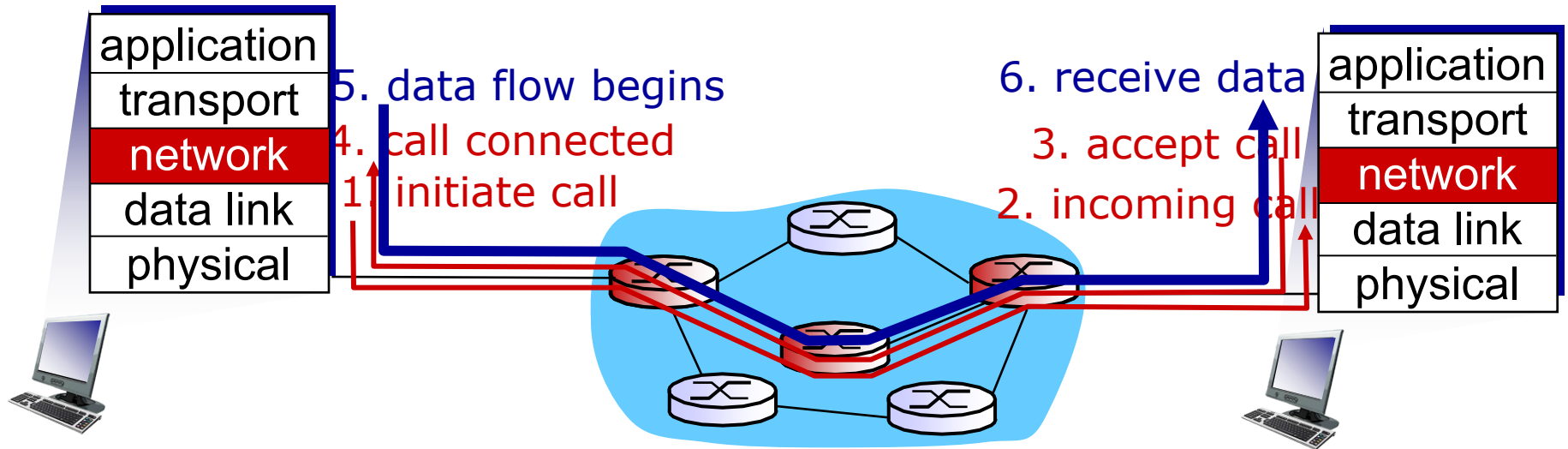
*forwarding table in
no*

Incoming int			ing VC #
1	12	3	22
2	63	1	18
3	7	2	17
1	97	3	87
...	...	...	...

VC routers maintain connection state info

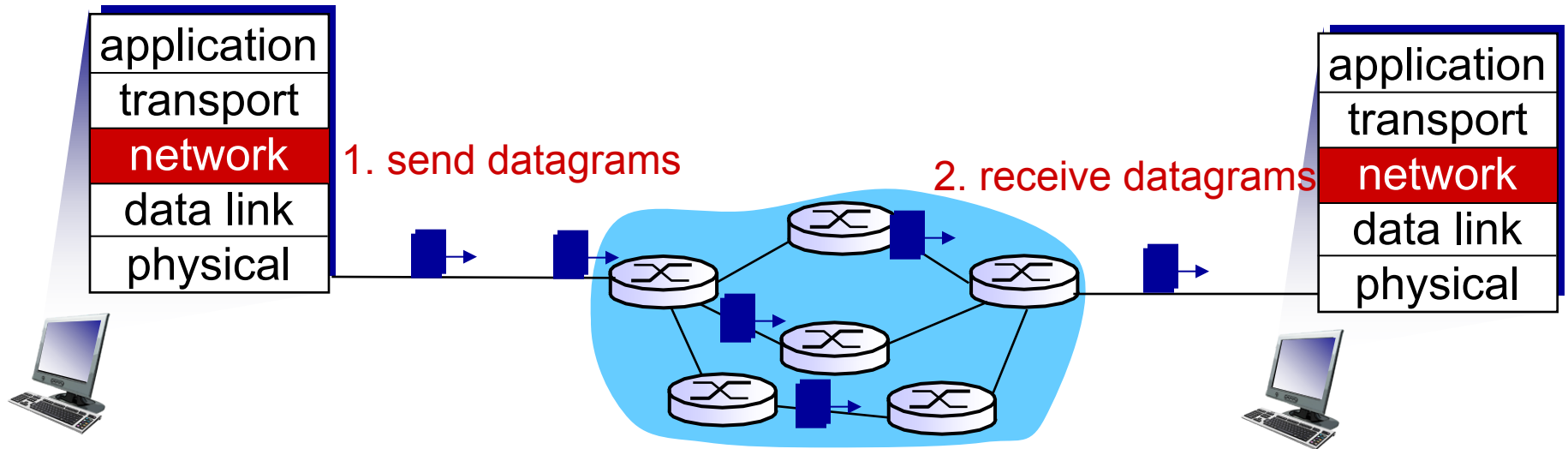
Virtual circuits: signaling protocols

- ❖ used to setup, maintain teardown VC
- ❖ used in ATM, frame-relay, X.25

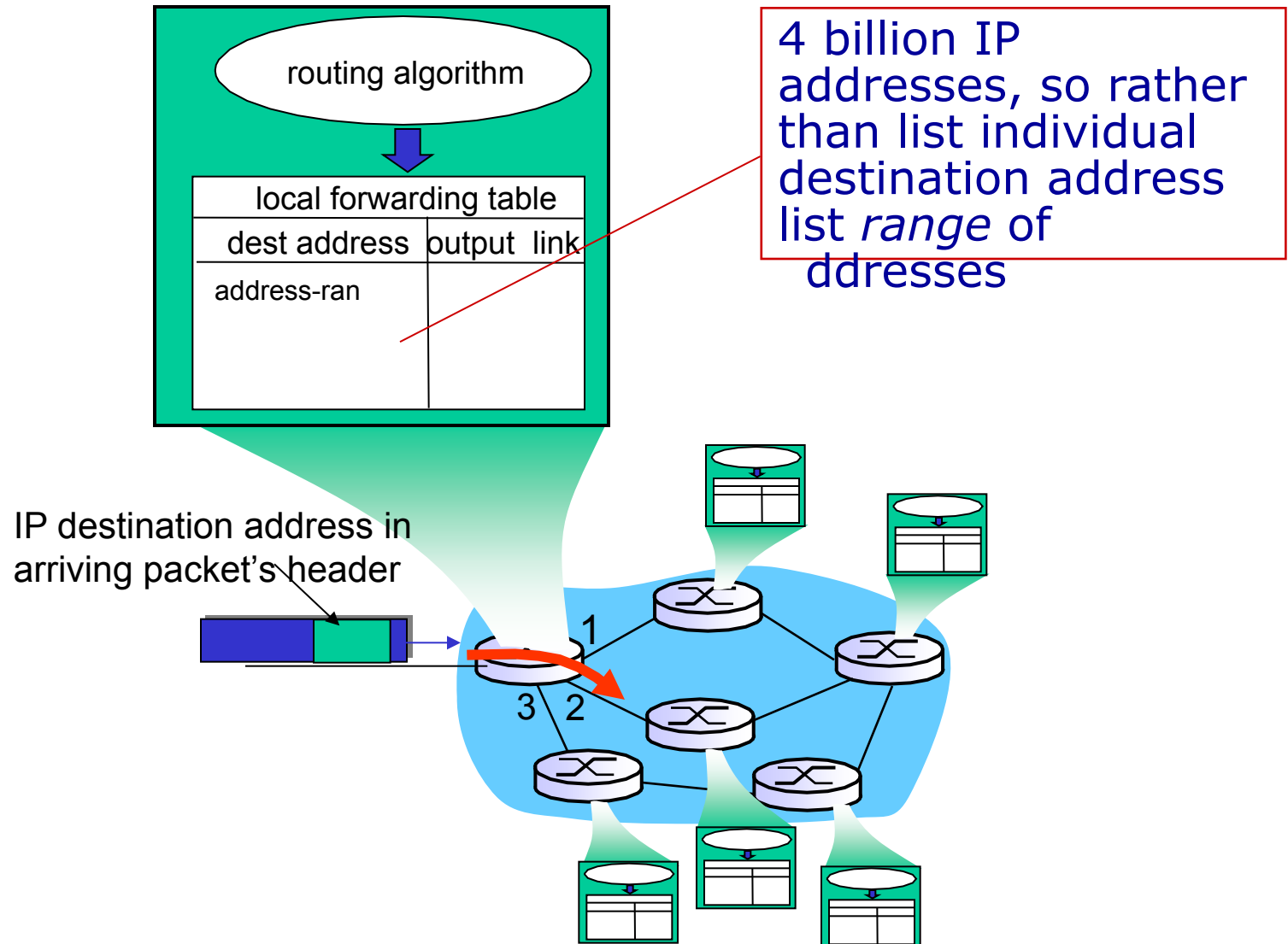


Datagram networks

- ❖ no call setup at network layer
- ❖ routers: no state about end-to-end connections
 - no network- "connection"



Datagram forwarding table



Datagram forwarding table

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 0	0
through 11001000	
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Q: but what happens if ranges don't divide up so nicely

Longest prefix matching

longest prefix matching

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

11001	
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

DA: 11001000 00010111 00010110 10100001

which interface?

DA: 11001000 00010111 00011000 10101010

which interface?

Datagram or VC network: why?

Internet (datagram)

- ❖ data exchange among computers
 - “elastic” service
 - different
 - uniform service difficult
- ❖ “smart” end systems (computers)
 - can adapt, perform control, error recovery
 - ***simple inside network, complexity at “edge”***

ATM (VC)

- ❖ evolved from telephony
 - an conversation: strict timing, guaranteed
- ❖ “dumb” end systems
 - telephones
 - ***complexity inside network***

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and
datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

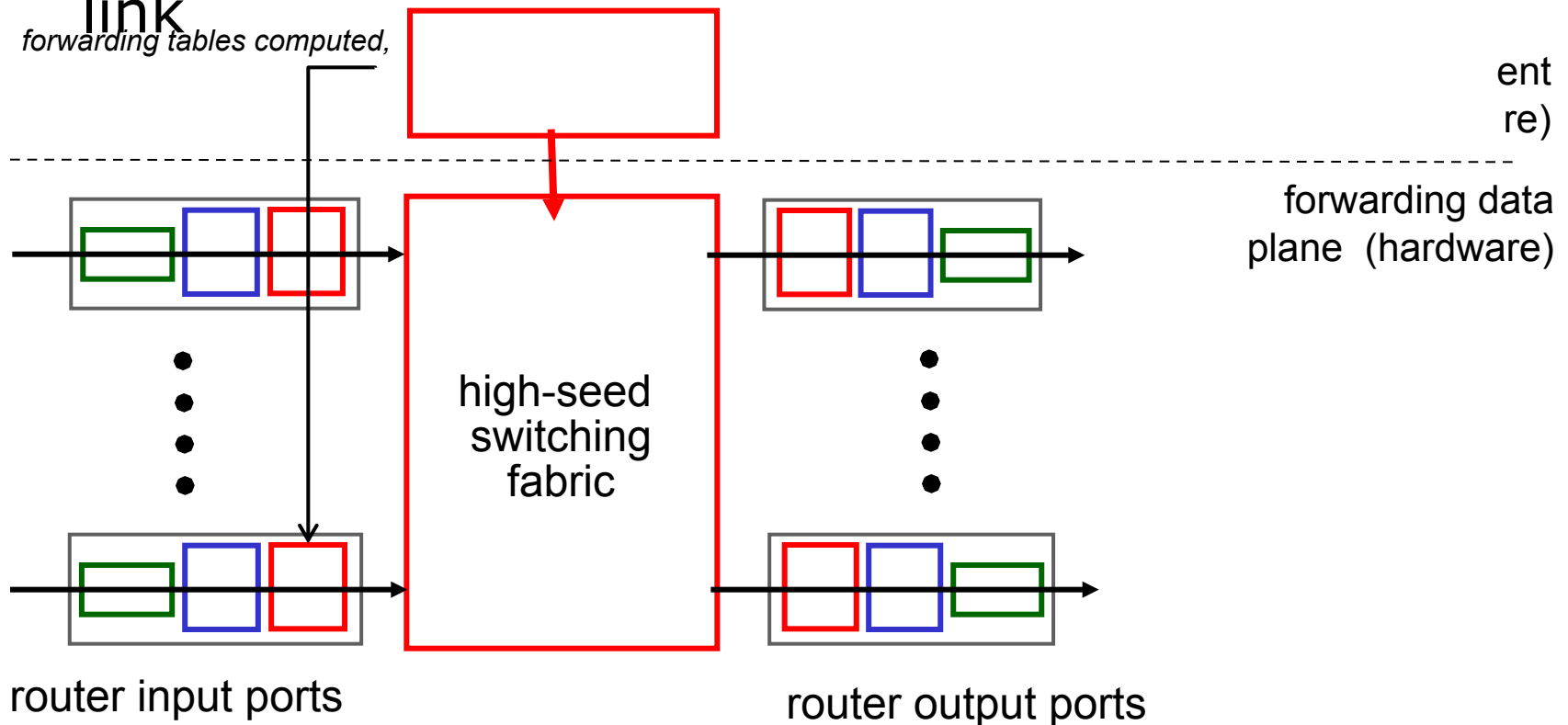
- OSPF
- BGP

4.7 broadcast and
multicast routing

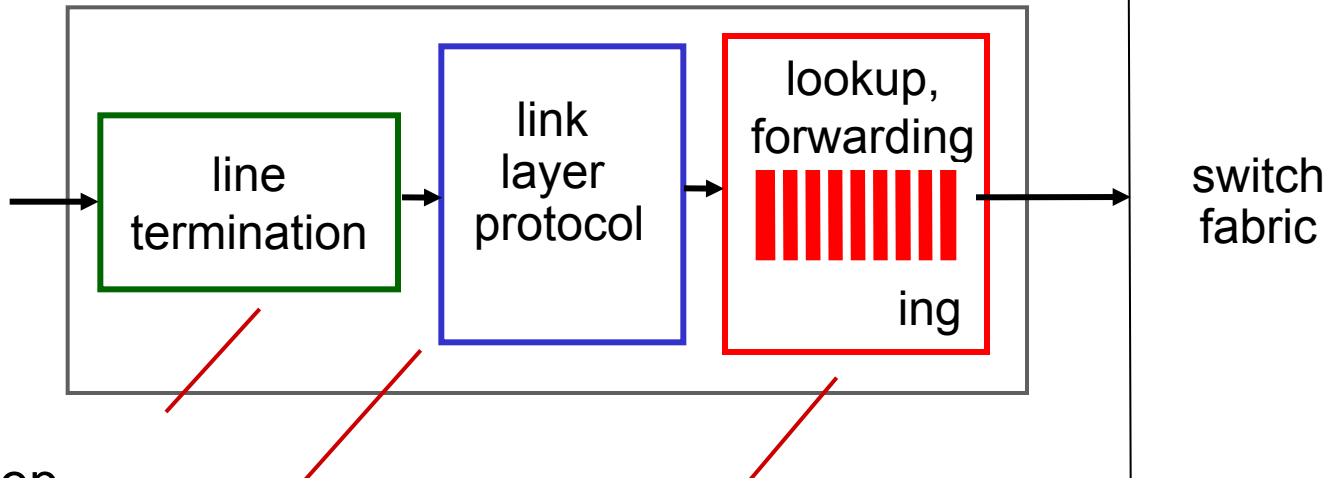
Router architecture overview

two key router functions:

- ❖ run routing algorithms/protocol (RIP, OSPF, BGP)
- ❖ *forwarding* datagrams from incoming to outgoing link



Input port functions



bit-level recep

data link layer:
e.g., Ethernet
see chapter 5

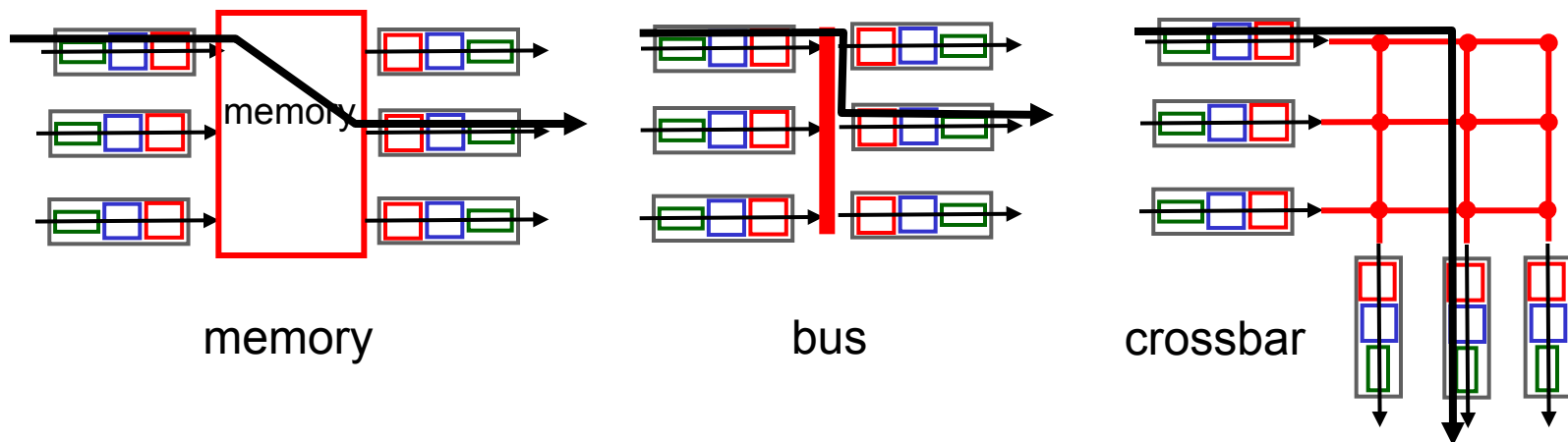
decentralized switching:

- ❖ given datagram dest., lookup output port using forwarding table in input port memory ("*match plus action*")
- ❖ goal: complete input port processing at 'line speed'
- ❖ queuing: if datagrams arrive faster than forwarding rate into switch fabric

Switching fabrics

- ❖ transfer packet from input buffer to appropriate output buffer
- ❖ switching rate: rate at which packets can be transferred to outputs

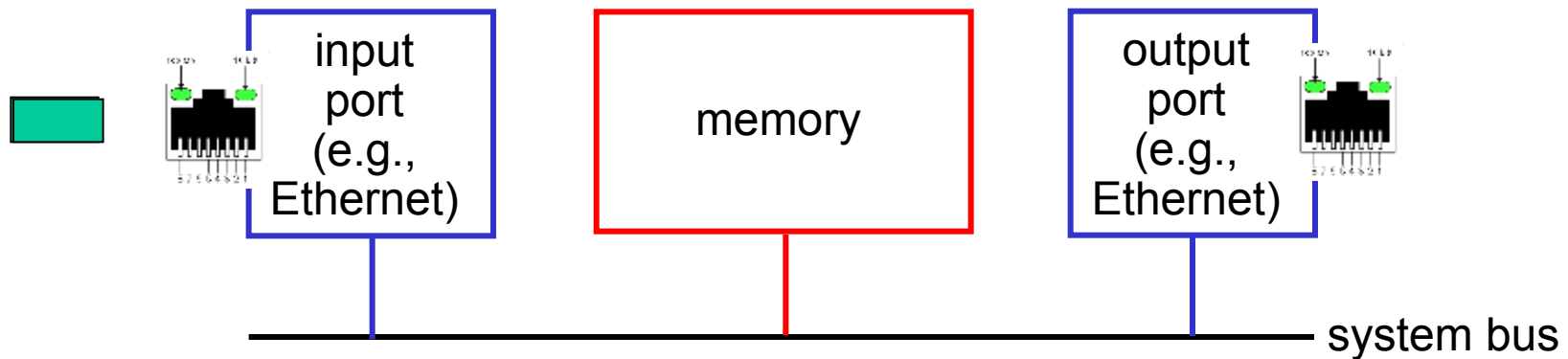
❖ three



Switching via memory

first generation routers:

- ❖ traditional computers with switching under direct control of CPU
- ❖ packet copied to memory

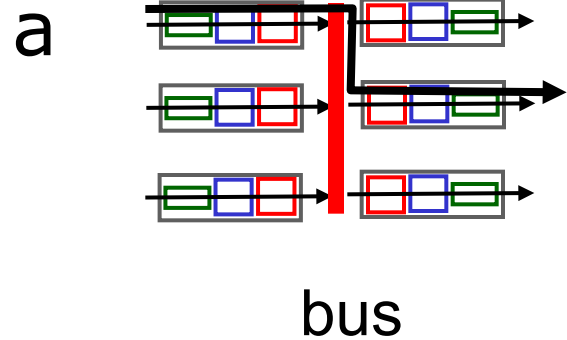


Switching via a bus

- ❖ datagram from input port memory to output p

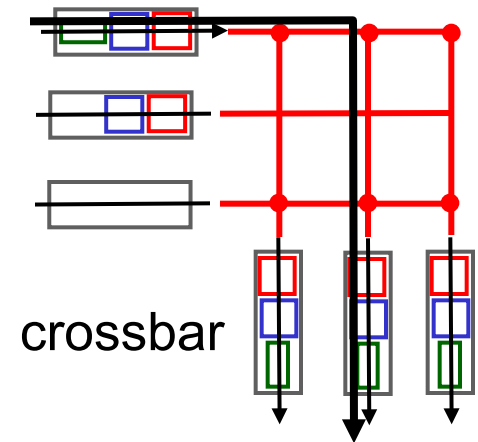
speed
bandwidth

- ❖ 32 Gbps bus, Cisco 5600:
sufficient speed for access
and enterprise routers



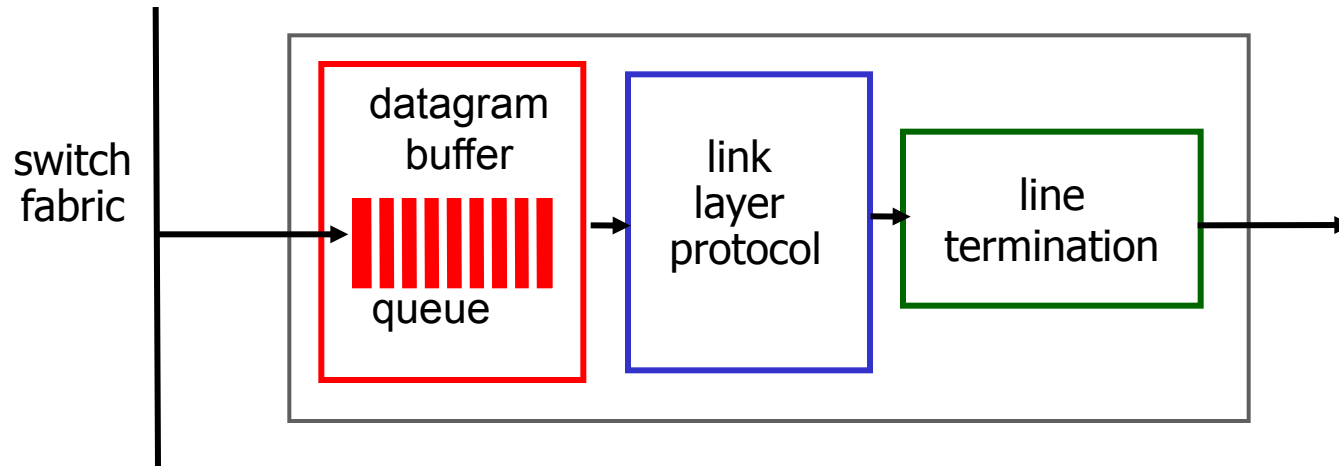
Switching via interconnection network

- ❖ overcome bus bandwidth limitations
- ❖ banyan networks, crossbar, other intercon
- ❖ advanced design: fragmenting datagram into fixed length cells, switch cells through the fabric.
- ❖ Cisco 12000: switches 60 Gbps through the interconnection network



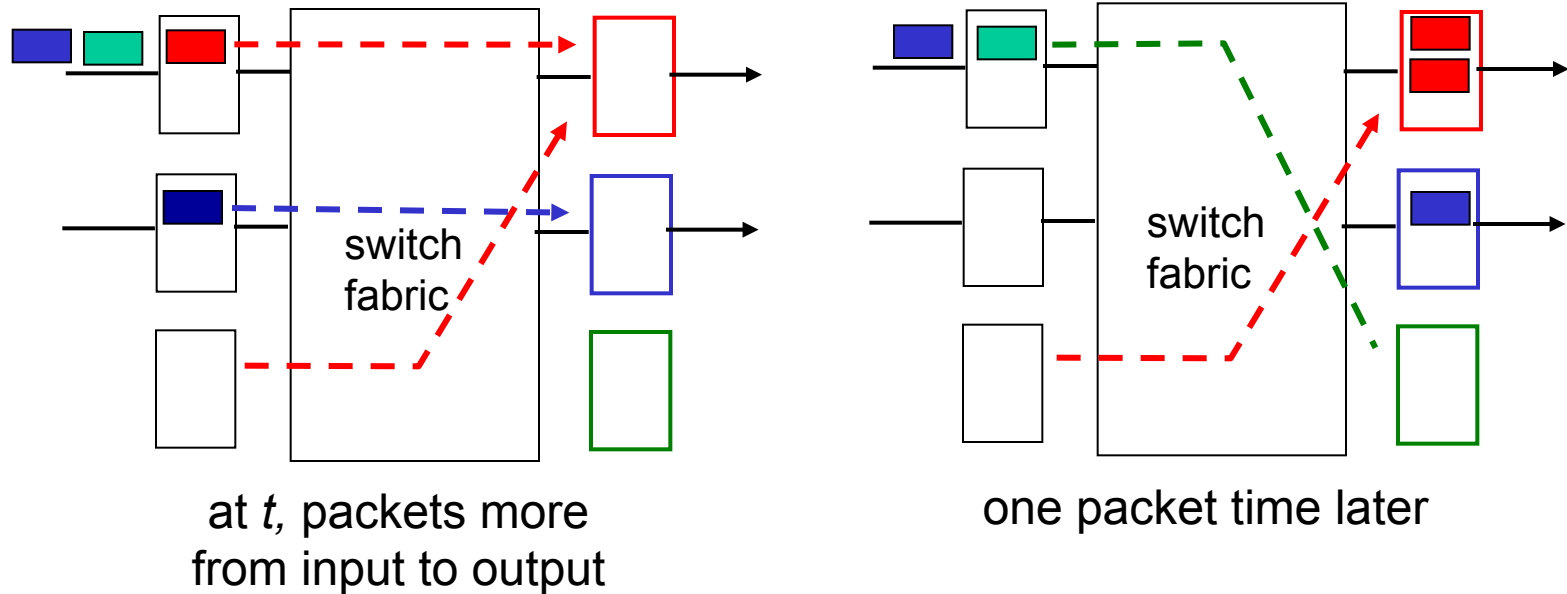
Output ports

This slide is HUGELY important!



- ❖ *buffering* requires buffers. Datagram (packets) can be lost if they arrive from fabric faster than the transmission rate due to congestion, lack of buffers
- ❖ *scheduling* is required to manage contention among competing flows for transmission. Priority scheduling – who gets best performance, network neutrality

Output port queueing



- ❖ buffering when arrival rate via switch exceeds output line speed
- ❖ *queueing (delay) and loss due to output port buffer overflow!*

How much buffering?

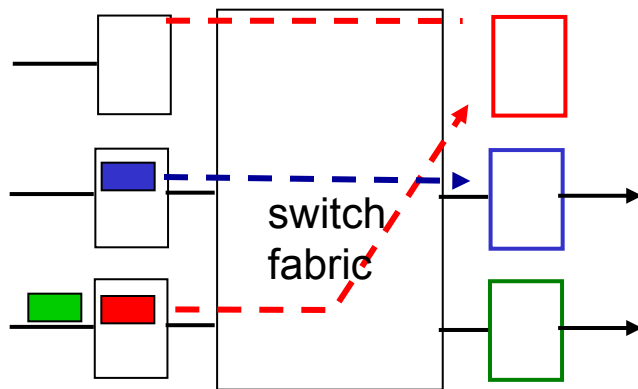
- ❖ RFC 3439 rule of thumb: average buffering equal to “typical” RTT (say 250 msec) times capacity C

buffer

$$\frac{RTT \cdot C}{\sqrt{N}}$$

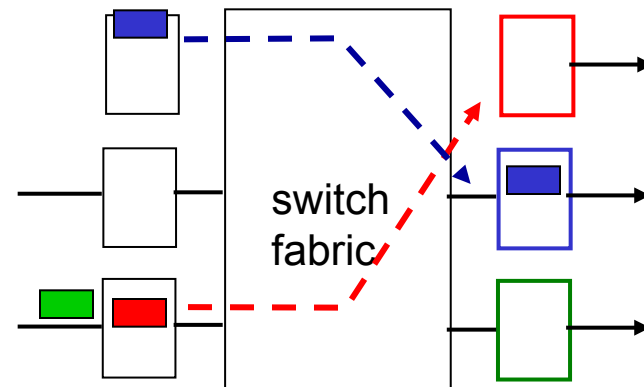
Input port queuing

- ❖ fabric slower than input ports combined -> queueing may occur at input queues
 - *queueing delay and loss due to input buffer overflow!*
- ❖ Head-of-the-Line



output port contention:
only one red datagram can
be transferred.
lower red packet is blocked

ng: queued



one packet time
later: green
packet
experiences HOL
blocking

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and
datagram netw

4.4 IP: I Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

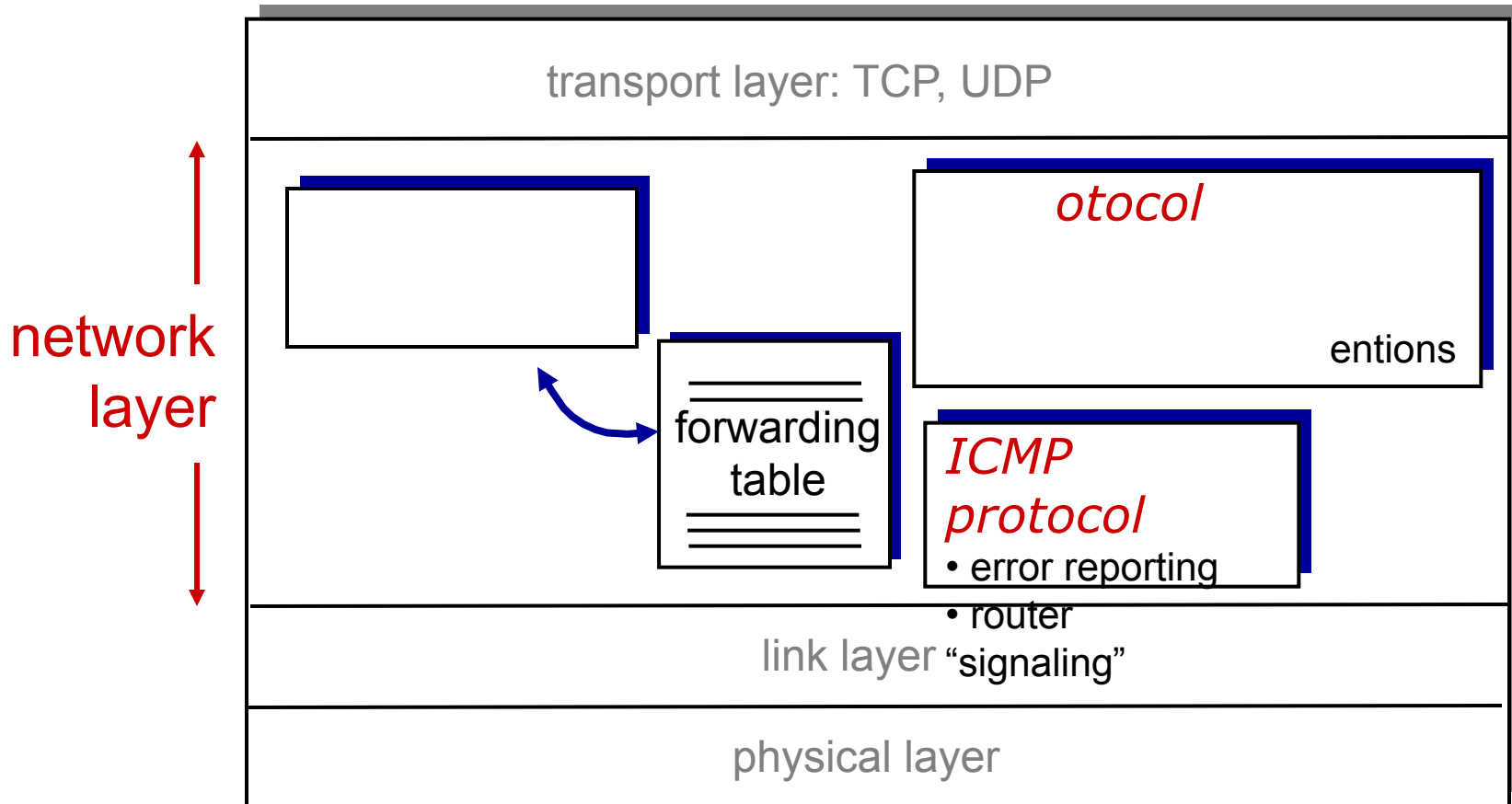
- link state
- distance vector
archical routing

- OSPF
- BGP

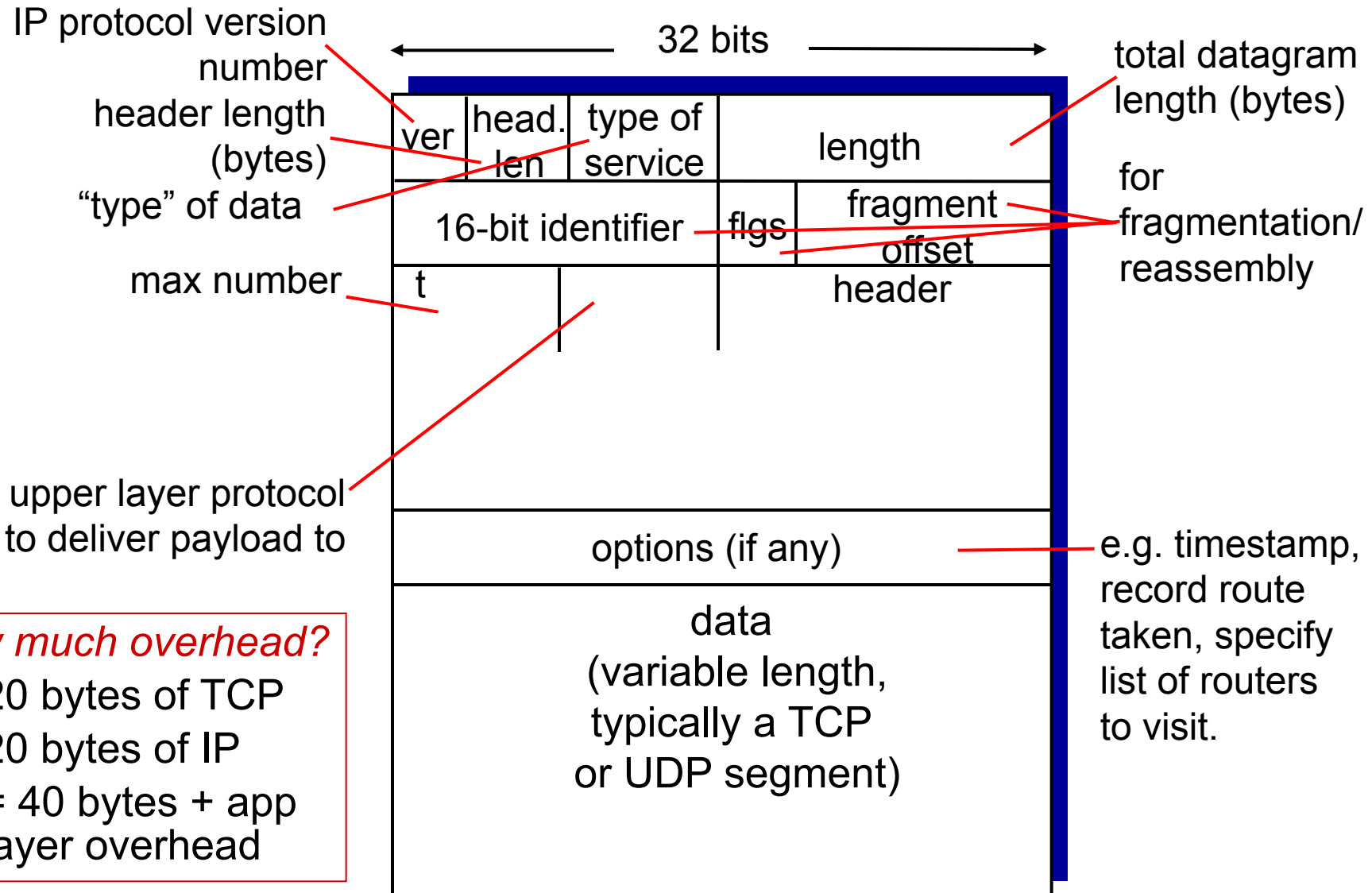
4.7 broadcast and
multicast routing

The Internet network layer

host, router network layer functions:



IP datagram format

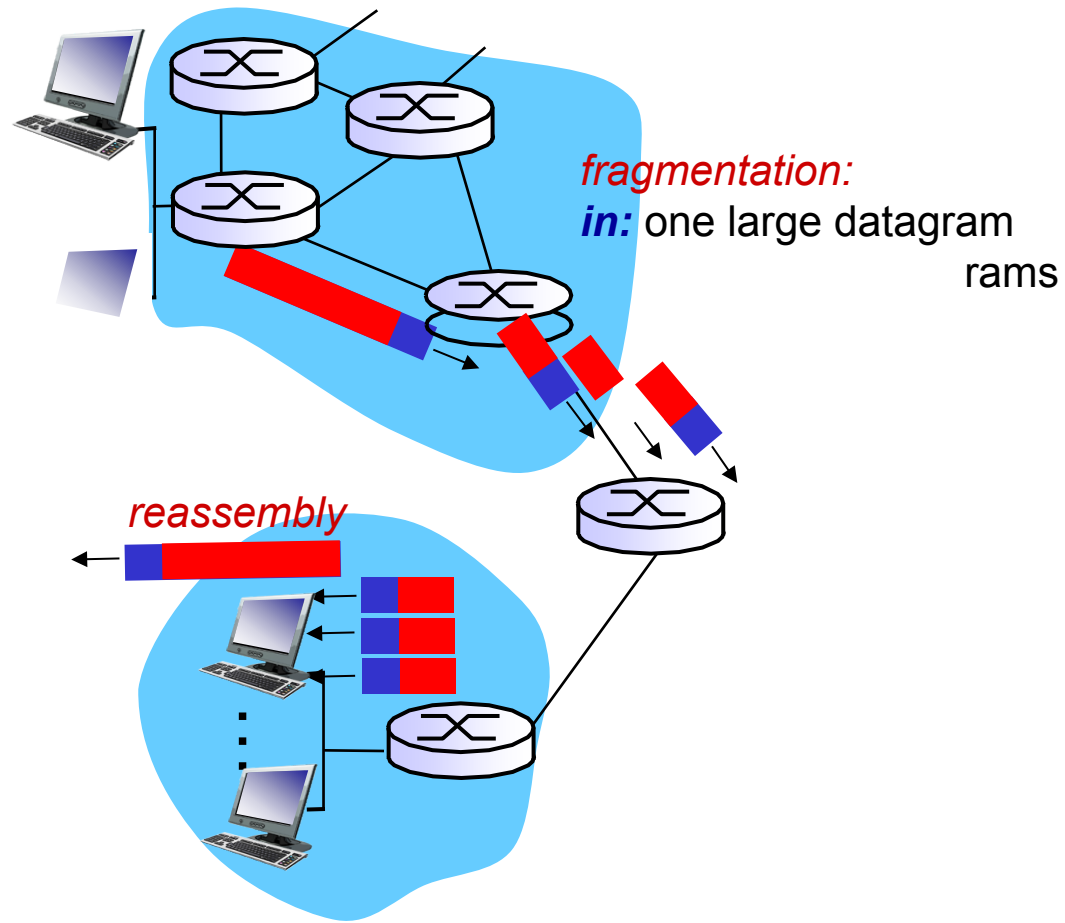


how much overhead?

- ❖ 20 bytes of TCP
- ❖ 20 bytes of IP
- ❖ = 40 bytes + app layer overhead

IP fragmentation, ~~reassembly~~

- ❖ network links have MTU (max.transfer size) - largest possible link-level



type
MTUs

- ❖ large IP datagram divided ("fragmented") within net
 - one datagram becomes several datagrams
 - "reassembled" only at final

IP fragmentation, ~~reassembly~~

example:

- ❖ 4000 byte datagram
- ❖

data file

offset =
1480/8

	length =4000	ID =x	fragflag =0	offset =0	
--	-----------------	----------	----------------	--------------	--

*becomes
grams*

				t	
--	--	--	--	---	--

	length =1500	ID =x	fragflag =1	offset =185	
--	-----------------	----------	----------------	----------------	--

	length =1040	ID =x	fragflag =0	offset =370	
--	-----------------	----------	----------------	----------------	--

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

- OSPF
- BGP

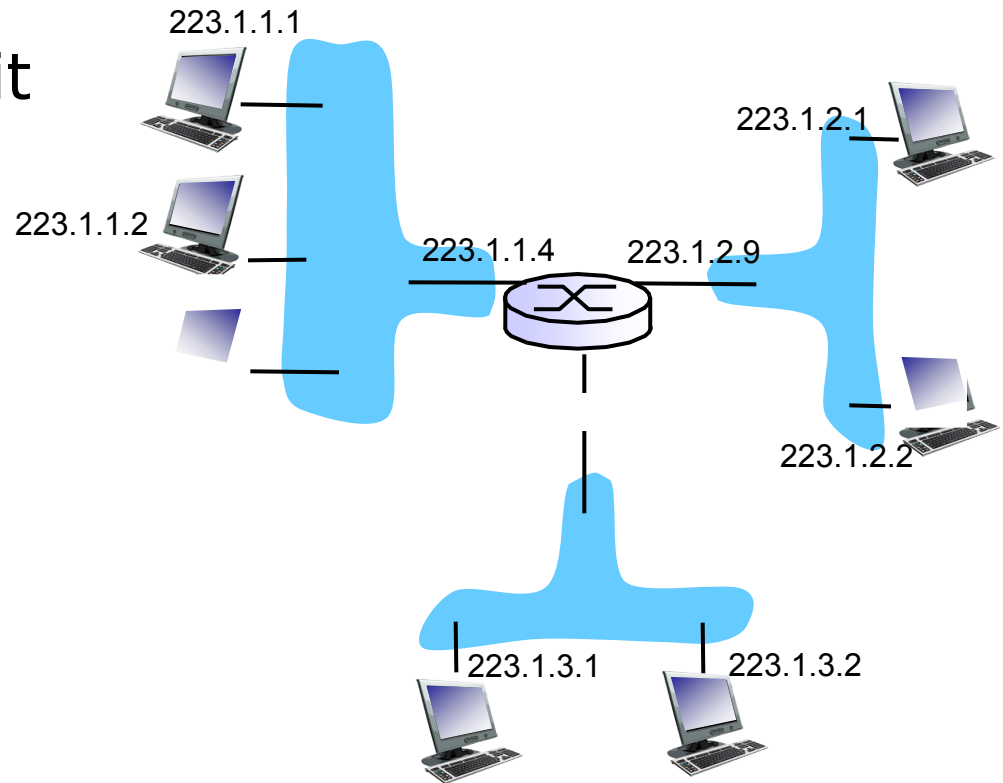
4.7 broadcast and multicast routing

IP addressing: introduction

- ❖ *IP address*: 32-bit identifier for host, router *interface*
- ❖ *interface*:

physic

- router's typically have multiple interfaces
- host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



- ❖ *IP addresses associated with each interface*

$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$

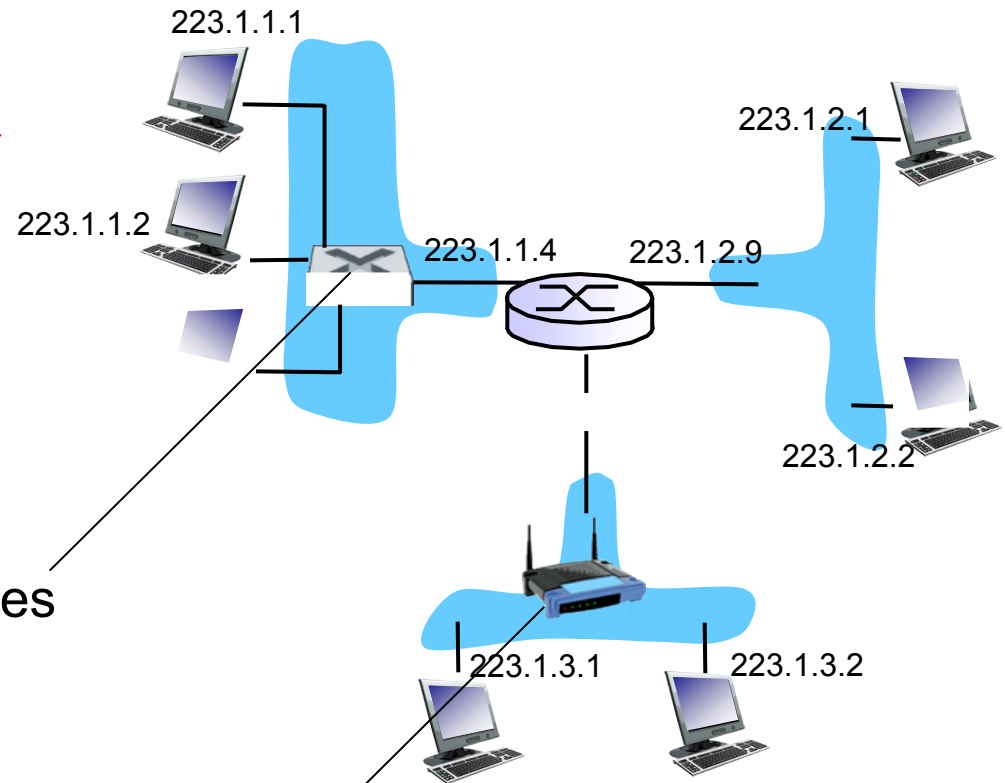
IP addressing: introduction

Q: how are interfaces actually connected?

t
6.

A: wired Ethernet interfaces connected by Ethernet switches

For now: don't need to worry about how one interface is connected to another (with no intervening router)

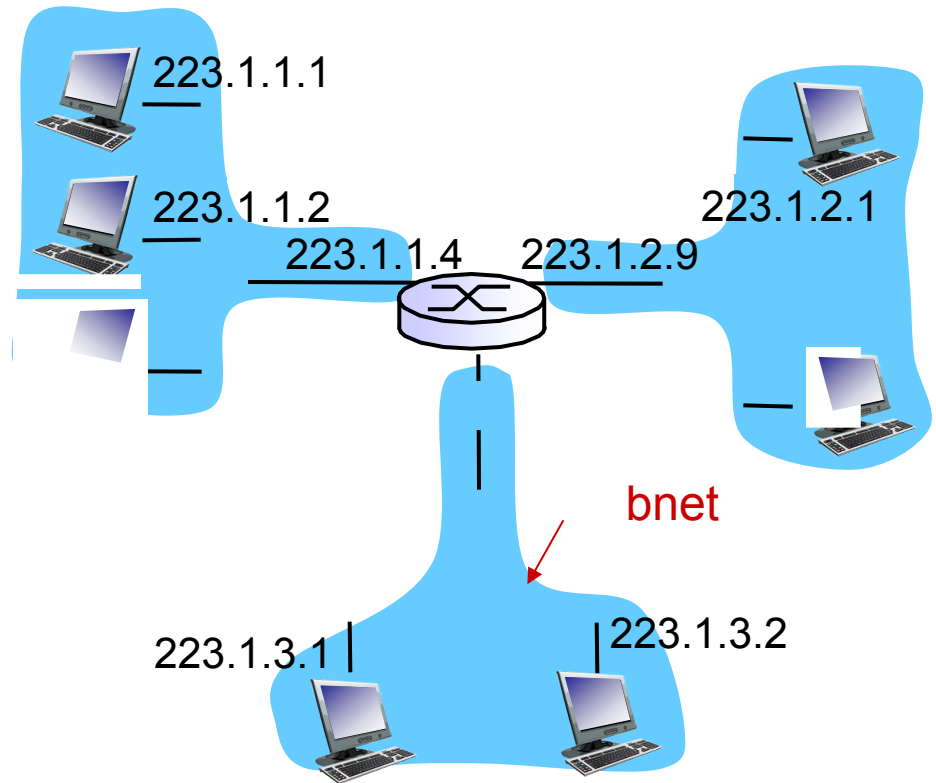


A: wireless WiFi interfaces connected by WiFi base station

Subnets

❖ IP address:

- subnet part - high order bits
 - host part - low
-
- device with same subnet part of IP address
 - can physically reach each other *without intervening router*



network consisting of 3 subnets

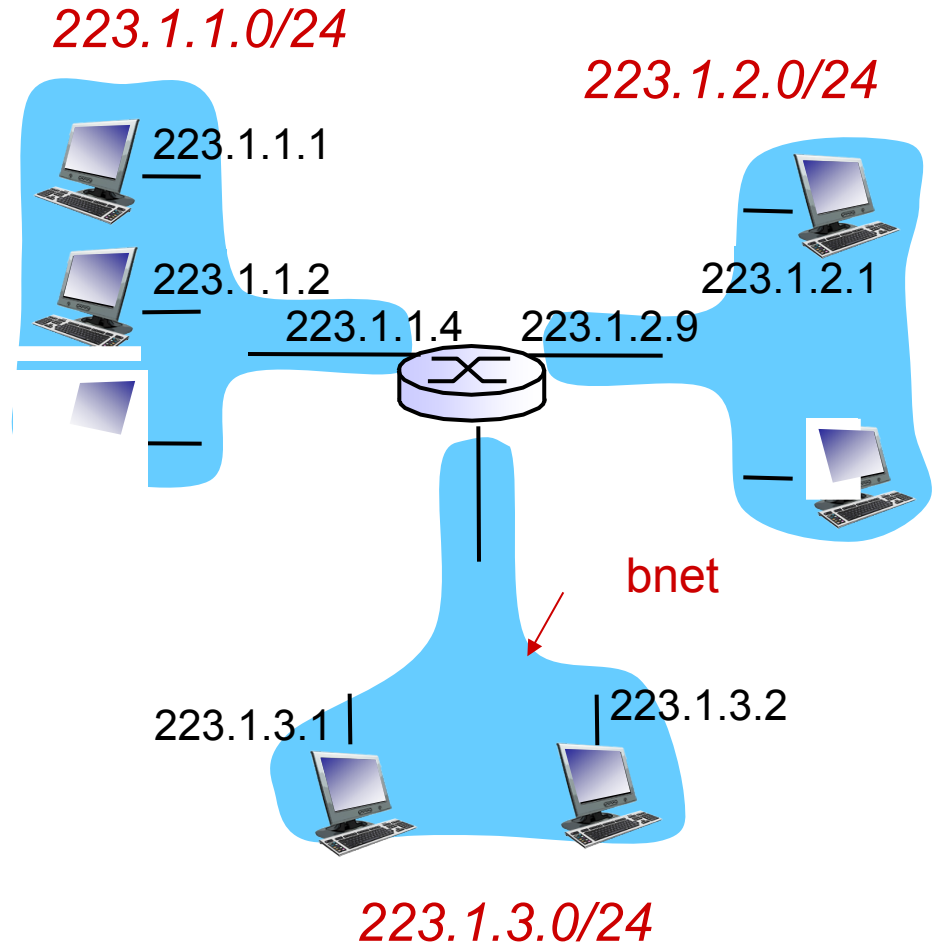
Subnets

recipe

- ❖ to determine the subnets, data

route
islands of isolated
networks

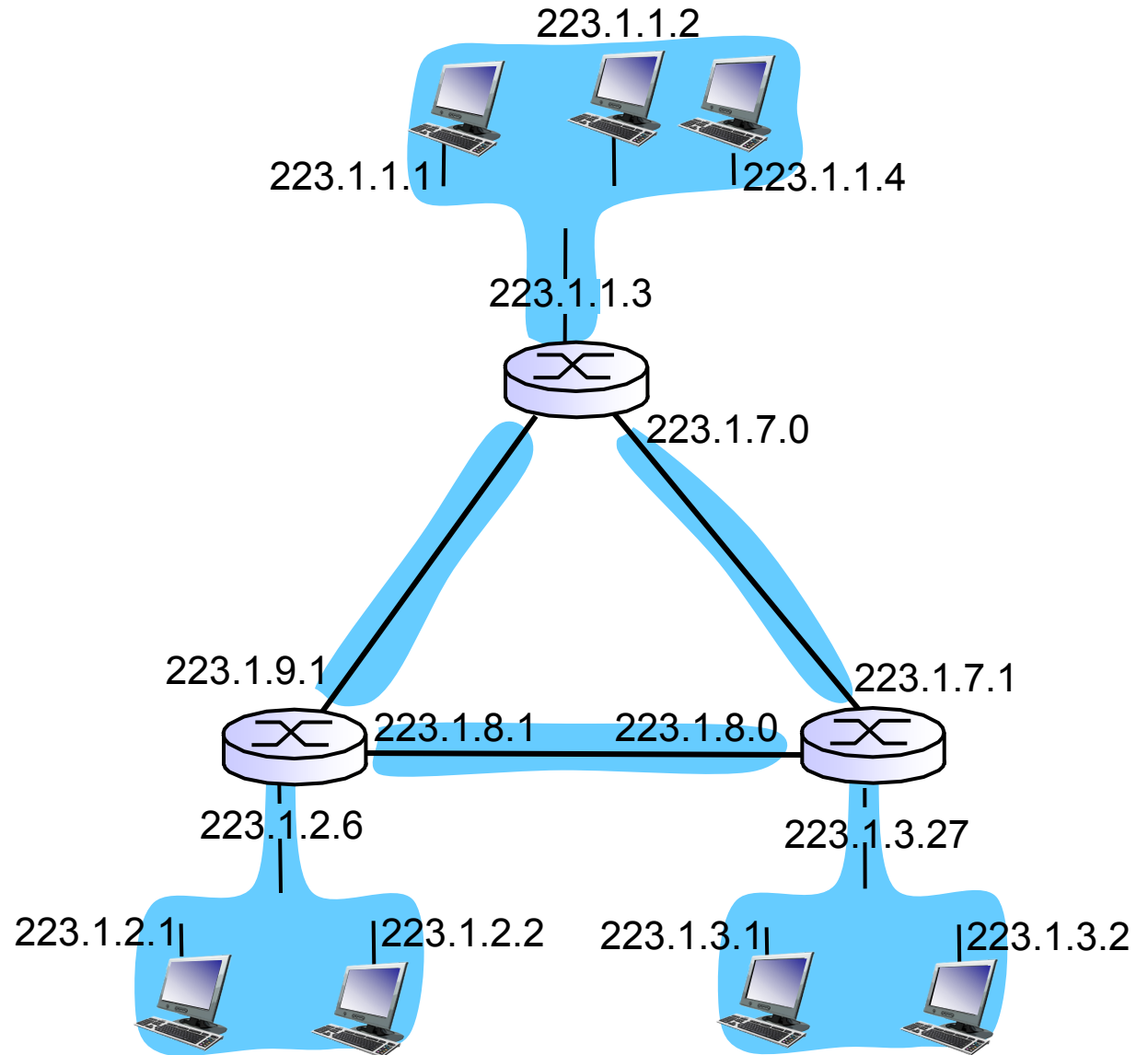
- ❖ each isolated network is called a *subnet*



subnet mask: /24

Subnets

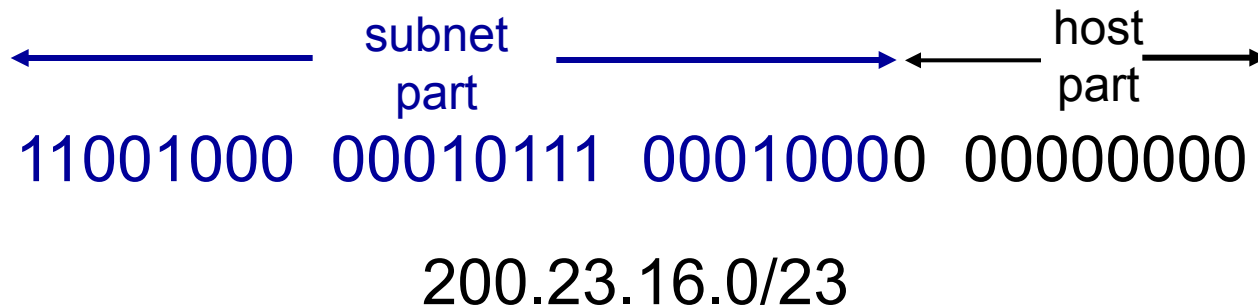
how many?



IP addressing: CIDR

CIDR: Classless InterDomain Routing

- subnet portion of address of arbitrary length



IP addresses: how to get ~~one?~~

Q: How does a *host* get IP address?

- ❖ hard-coded b in in a file
 - UN
- ❖ **DHCP:** Dynamic Host Configuration Protocol: dynamically get address from as server
 - “plug-and-play”

DHCP: Dynamic Host Configuration Protocol

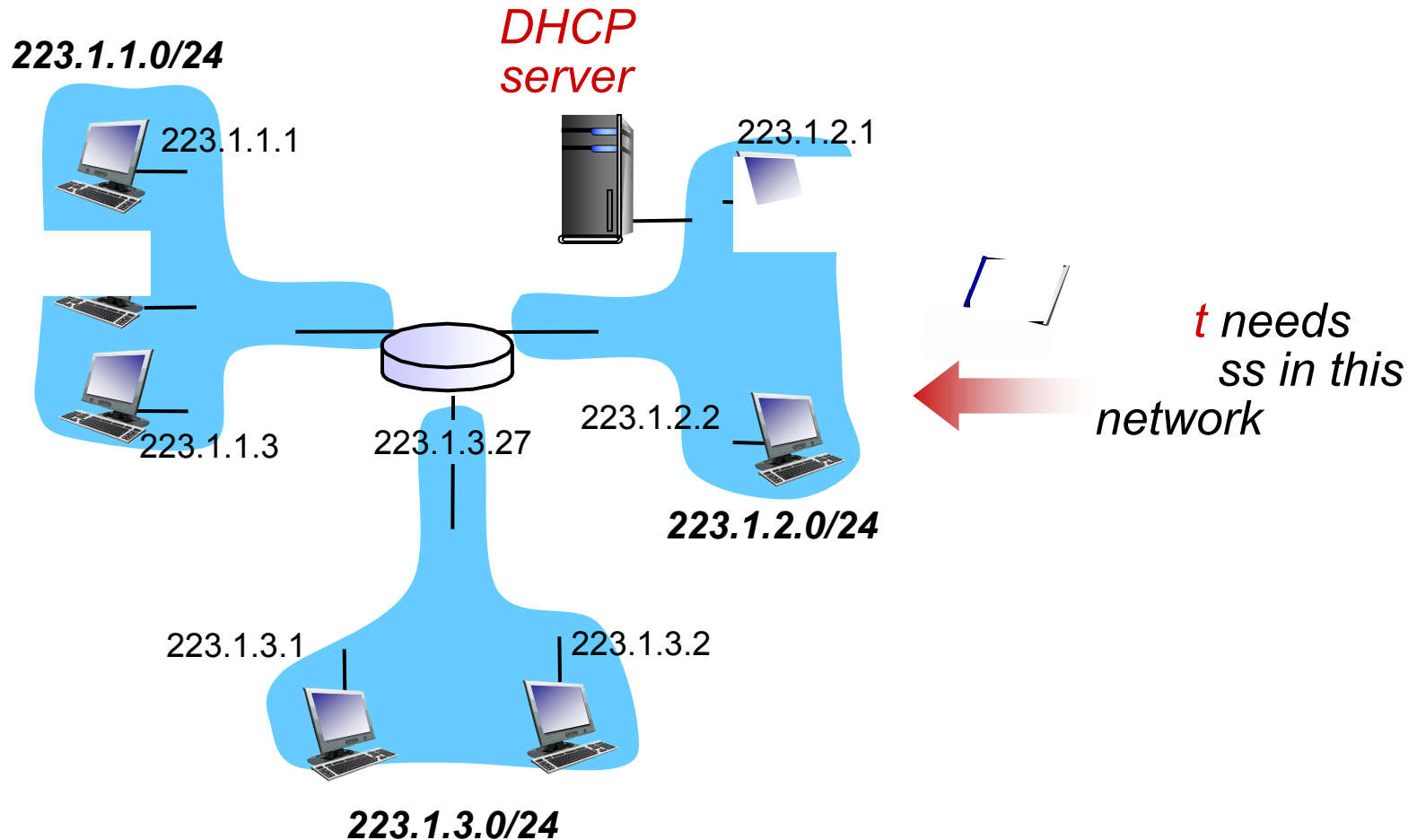
goal: allow host to *dynamically* obtain its IP address from network server when it joins network

- can renew its s in use
- sup oin
network (more shortly)

DHCP overview:

- host broadcasts "DHCP discover" msg [optional]
- DHCP server responds with "DHCP offer" msg [optional]
- host requests IP address: "DHCP request" msg
- DHCP server sends address: "DHCP ack" msg

DHCP client-server scenario

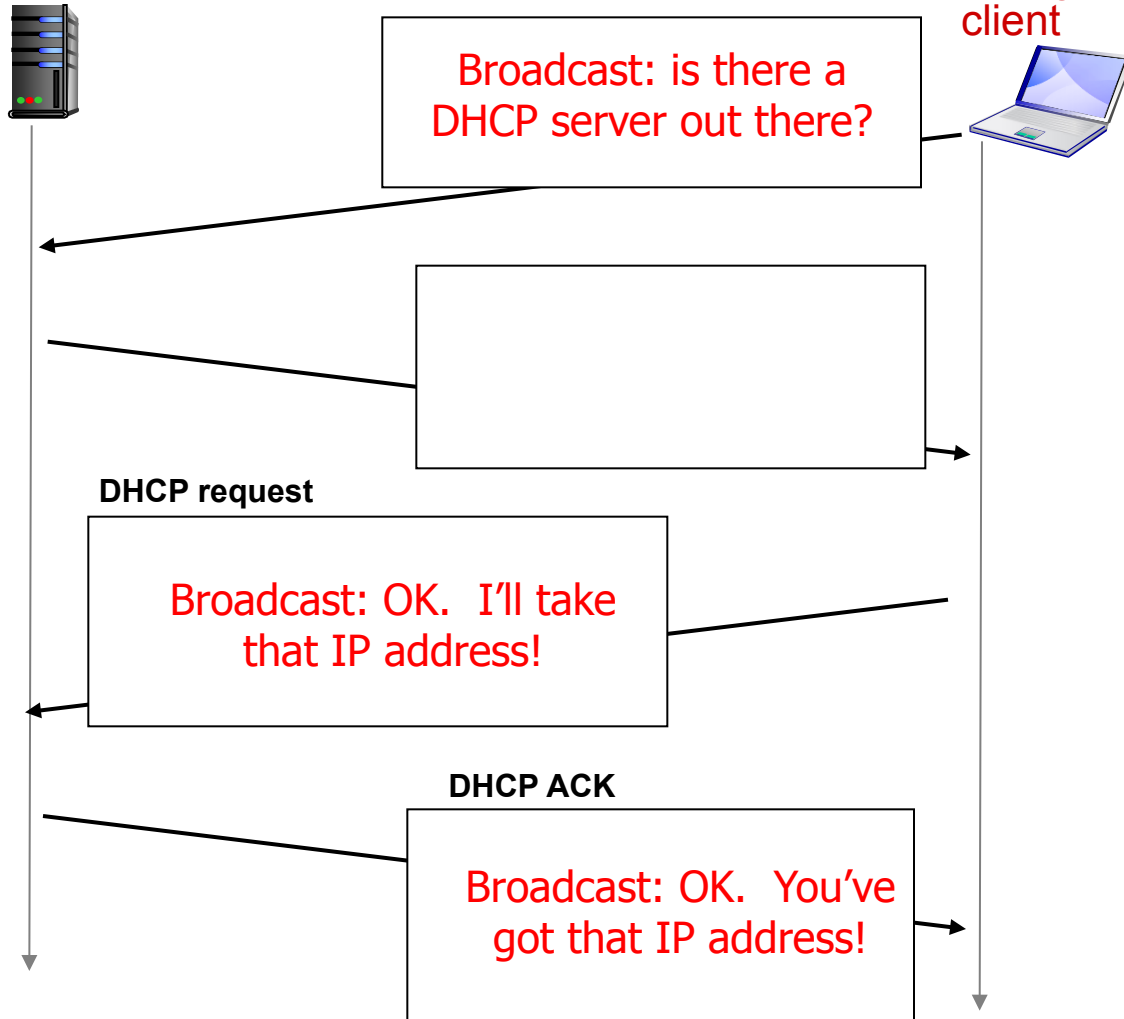


DHCP client-server scenario

DHCP server: 223.1.2.5

DHCP discover

arriving
client



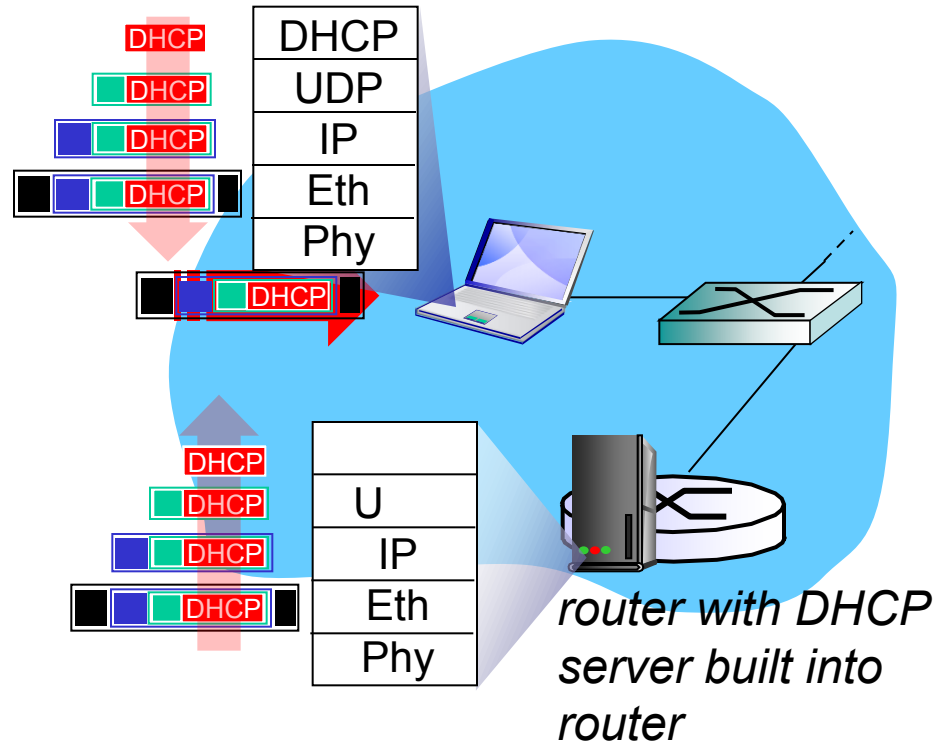
DHCP: more than IP addresses

DHCP can return more than just allocated
IP address on subnet:

- address of first client

host

DHCP: example

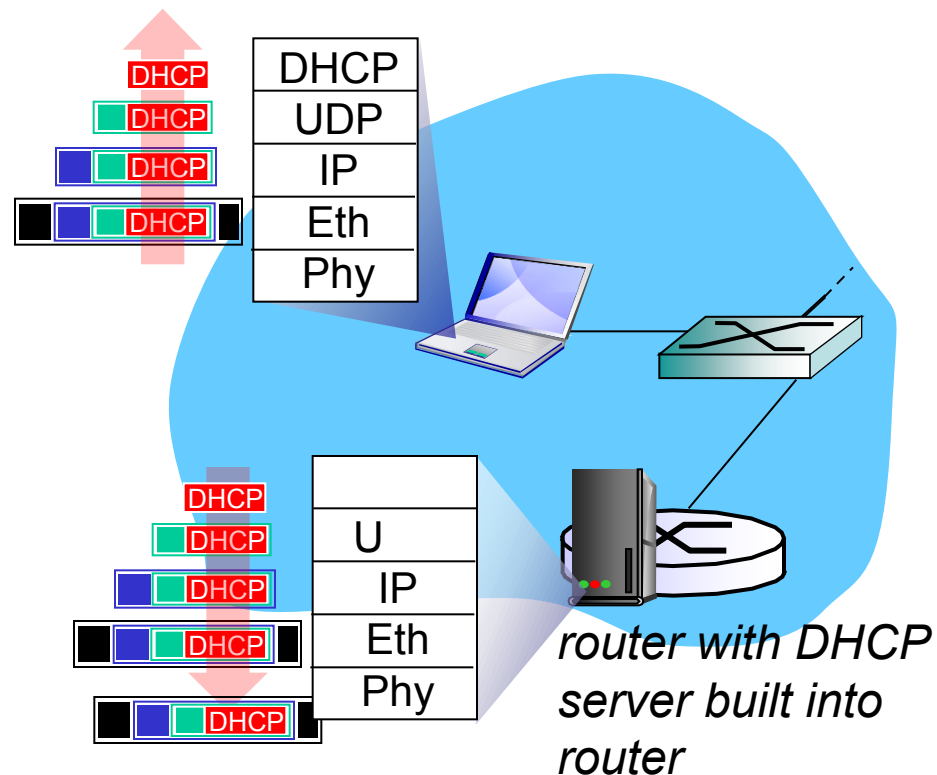


- ❖ connecting laptop needs its IP address, addr of first-hop router, dr of DNS server: CP request

in 802.1

- ❖ ~~Ethernet~~ Ethernet frame broadcast (dest: FFFFFFFFFFFFFFFF) on LAN, received at router running DHCP server
- ❖ Ethernet demuxed to IP demuxed, UDP demuxed to DHCP

DHCP: example



- ❖ DCP server formulates DHCP ACK containing client's IP address, IP address of first-hop router for

- ❖ client, demuxing up to DHCP at client
client now knows its IP address, name and IP address of DSN server, IP address of its first-hop router

DHCP: Wireshark output (home LAN)

Message type: **Boot Request (1)**

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 0.0.0.0 (0.0.0.0)

Your (c

Next s

Relay

Client MAC address:

Server host name not gi

Boot file name not given

Magic cookie: (OK)

Option: (t=53,l=1) **DHCP Message Type = DHCP Request**

Option: (61) Client identifier

Length: 7; Value: 010016D323688A;

Hardware type: Ethernet

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

Option: (t=50,l=4) Requested IP Address = 192.168.1.101

Option: (t=12,l=5) Host Name = "nomad"

Option: (55) Parameter Request List

Length: 11; Value: 010F03062C2E2F1F21F92B

1 = Subnet Mask; 15 = Domain Name

3 = Router; 6 = Domain Name Server

44 = NetBIOS over TCP/IP Name Server

.....

request

Message type: **Boot Reply (2)**

Hardware type: Ethernet

Hardware address length: 6

Hops: 0

Transaction ID: 0x6b3a11b7

Seconds elapsed: 0

Bootp flags: 0x0000 (Unicast)

Client IP address: 192.168.1.101 (192.168.1.101)

Your (client) IP address: 0.0.0.0 (0.0.0.0)

Next server IP address: 192.168.1.1 (192.168.1.1)

Relay agent IP address: 0.0.0.0 (0.0.0.0)

Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)

e not given

t given

reply

255.255.0
1

Length: 12; Value: 445747E2445749F244574092;

IP Address: 68.87.71.226;

IP Address: 68.87.73.242;

IP Address: 68.87.64.146

Option: (t=15,l=20) Domain Name = "hsd1.ma.comcast.net."

IP addresses: how to get one?

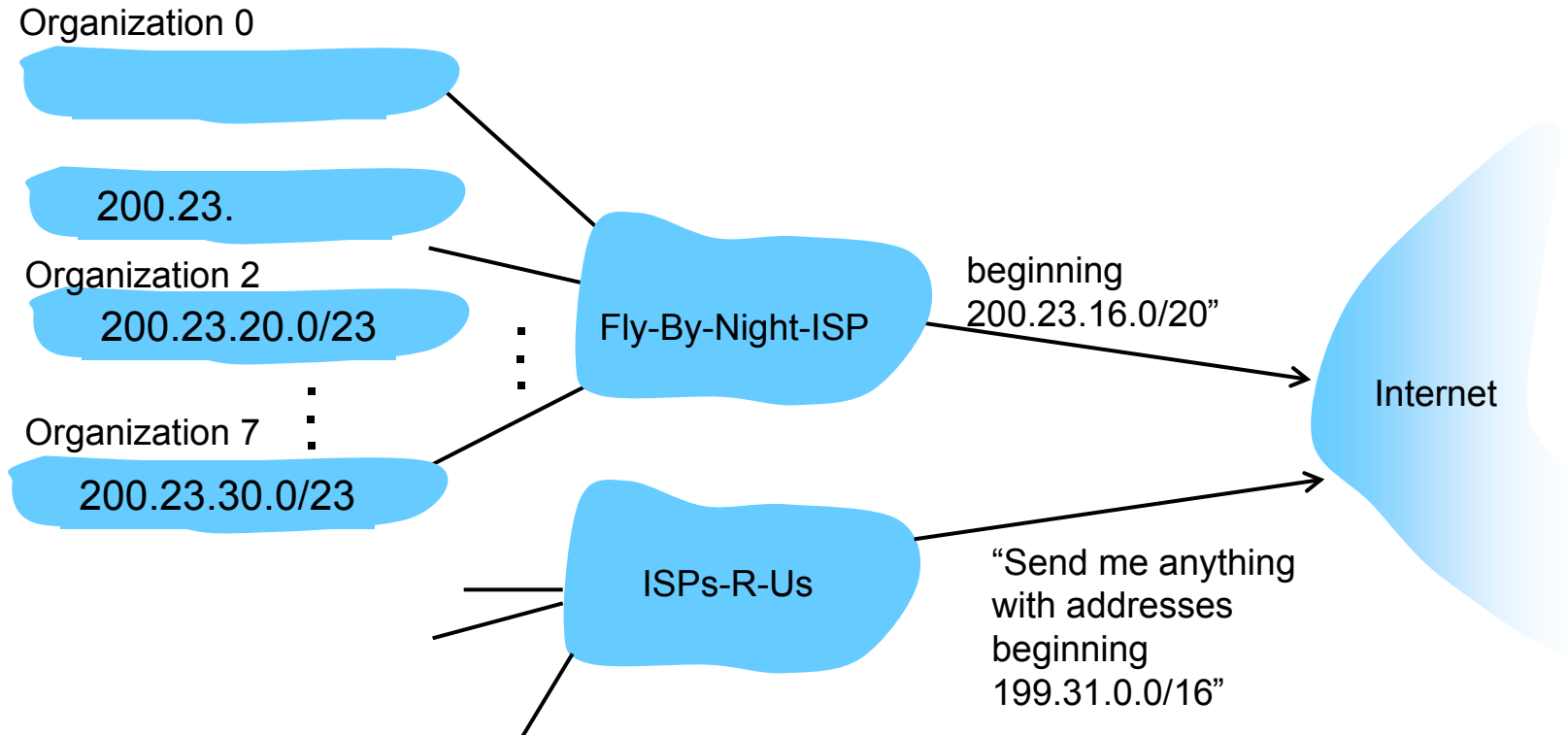
Q: how does *network* get subnet part of IP addr?

A: gets allocated portion of its provider ISP's address

ISP's block					0.23.16.0/20
Organization 0	<u>11001000</u>	<u>00010111</u>	<u>00010000</u>	00000000	200.23.16.0/23
Organization 1	<u>11001000</u>	<u>00010111</u>	<u>00010010</u>	00000000	200.23.18.0/23
Organization 2	<u>11001000</u>	<u>00010111</u>	<u>00010100</u>	00000000	200.23.20.0/23
...					
Organization 7	<u>11001000</u>	<u>00010111</u>	<u>00011110</u>	00000000	200.23.30.0/23

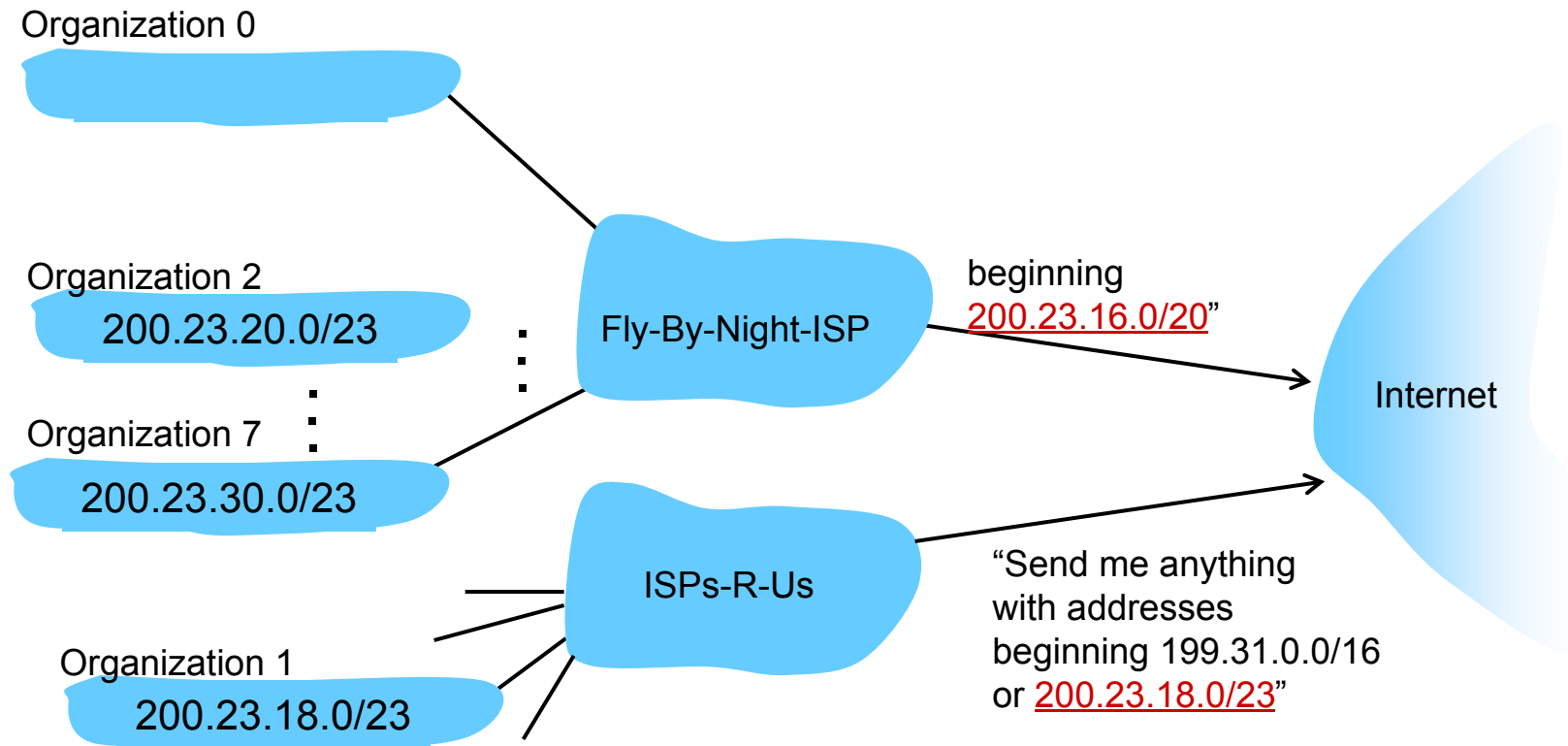
Hierarchical addressing: route aggregation

hierarchical addressing allows efficient advertisement of information:



Hierarchical addressing: more specific routes

ISPs-R-Us has a more specific route to Organization 1



IP addressing: the last word...

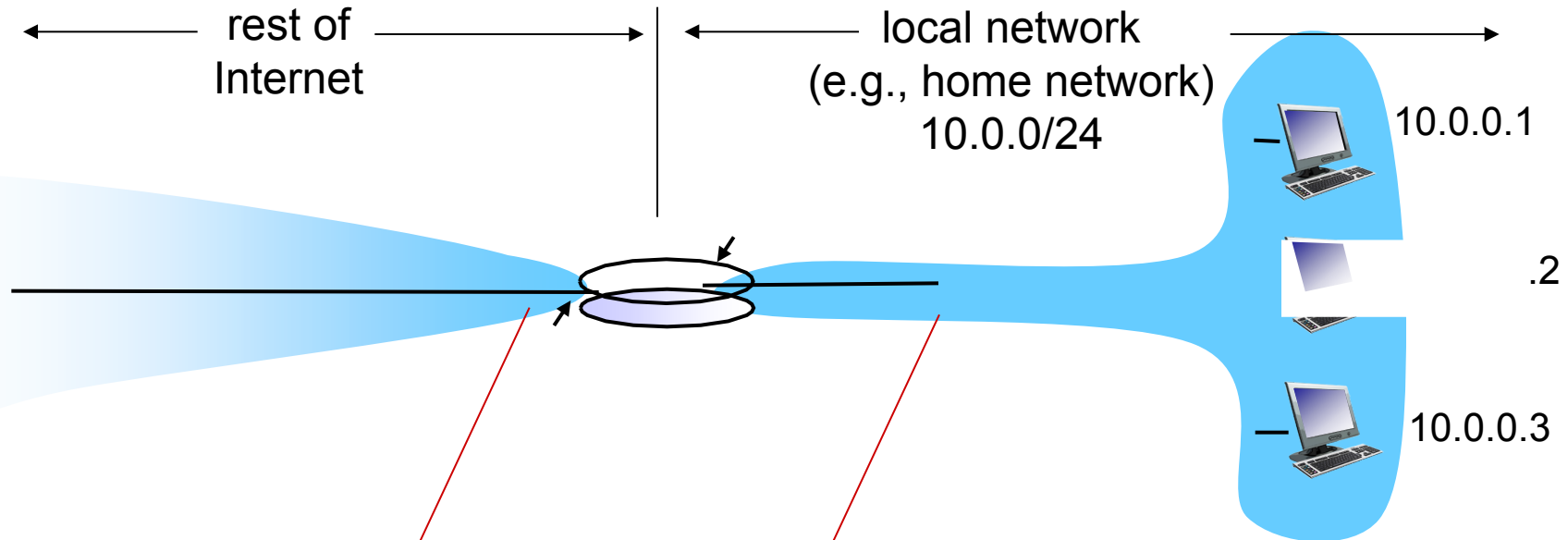
Q: how does an ISP get block of addresses?

A: ICANN: International Internet Corporation for Assigned Names and Numbers

http:

- allocates addresses
- manages DNS
- assigns domain names, resolves disputes

NAT: network address translation



all datagrams *leaving* local network have *same* single source NAT IP address: 138.76.29.7, different source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT: network address translation

motivation: local network uses just one IP address as far as outside world is concerned:

:

- can connect to outside world in local network without notifying outside world
- can change ISP without changing addresses of devices in local network
- devices inside local net not explicitly addressable, visible by outside world (a security plus)

NAT: network address translation

implementation: NAT router must:

- *outgoing datagrams: replace* (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #)

(N
addr

nation

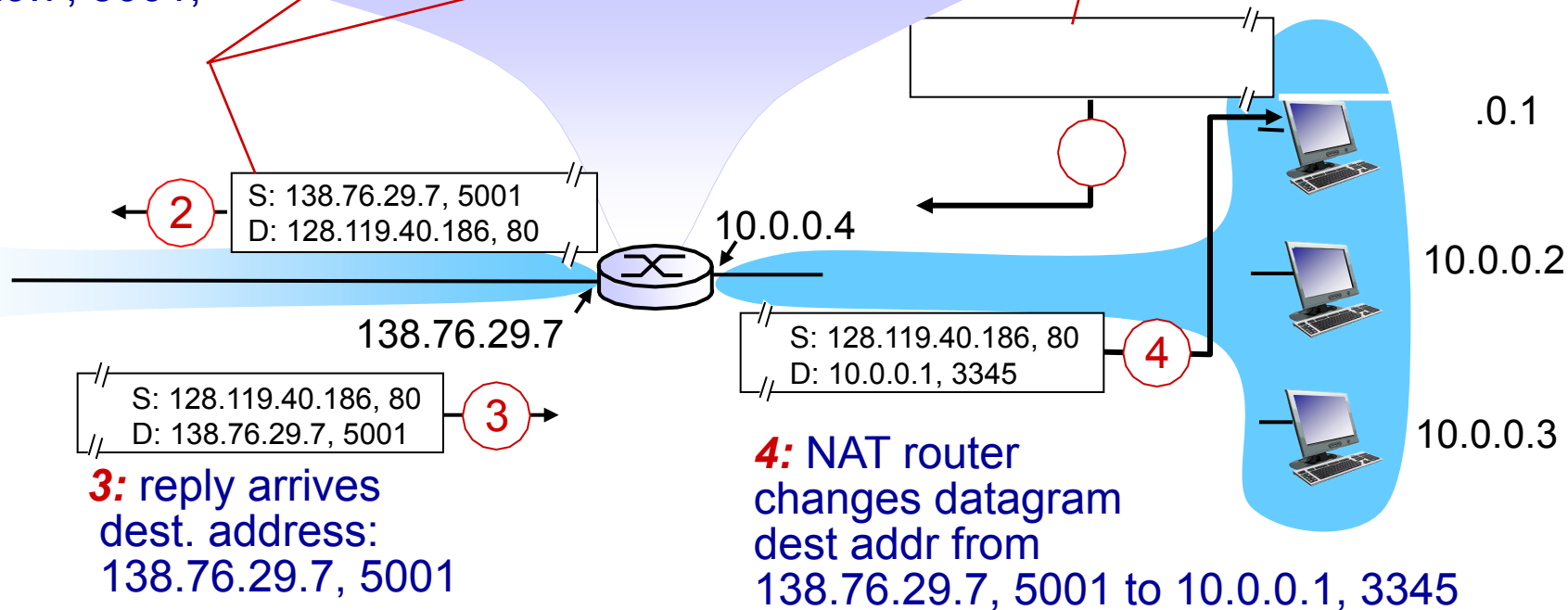
- *remember (in NAT translation table)* every (source IP address, port #) to (NAT IP address, new port #) translation pair
- *incoming datagrams: replace* (NAT IP address, new port #) in dest fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

NAT: network address translation

2: NAT router changes datagram source addr from 10.0.0.1, 3345 to 138.76.29.7, 5001, updat

NAT translation table	
WAN side addr	LAN side addr
138.76.29.7, 5001	10.0.0.1, 3345
.....	

1: host 10.0.0.1 sends datagram to 128.119.40.186, 80



NAT: network address translation

- ❖ 16-bit port-number field:
 - 60,000 simultaneous connections with a single address!
 -
 - layer 3
 - violates end-to-end argument
 - NAT possibility must be taken into account by app designers, e.g., P2P applications
 - address shortage should instead be solved by IPv6

NAT traversal problem

- ❖ client wants to connect to server with address 10.0.0.1

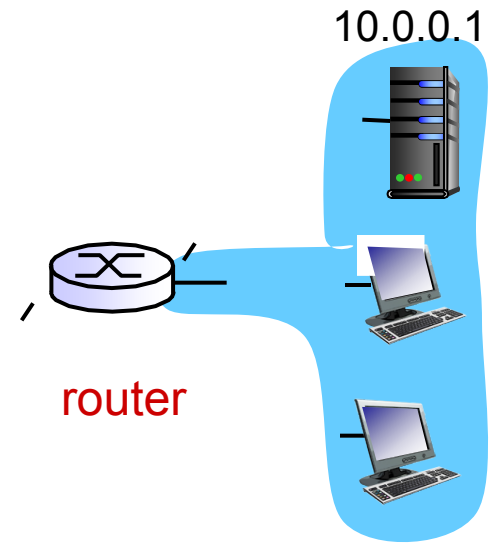
- server address local to LAN (cli

NAT
138.76.29.7

- ❖ ***solution1:*** statically configure NAT to forward incoming connection requests at given port to server
 - e.g., (123.76.29.7, port 2500) always forwarded to 10.0.0.1 port 25000

client

?

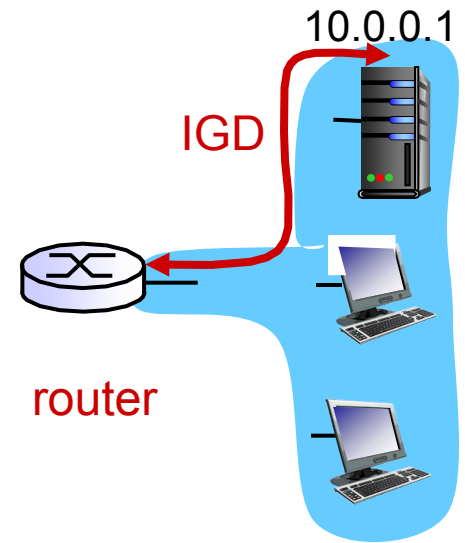


NAT traversal problem

- ❖ *solution 2:* Universal Plug and Play (UPnP) Internet Gateway Device (IGD) Protocol. Allow

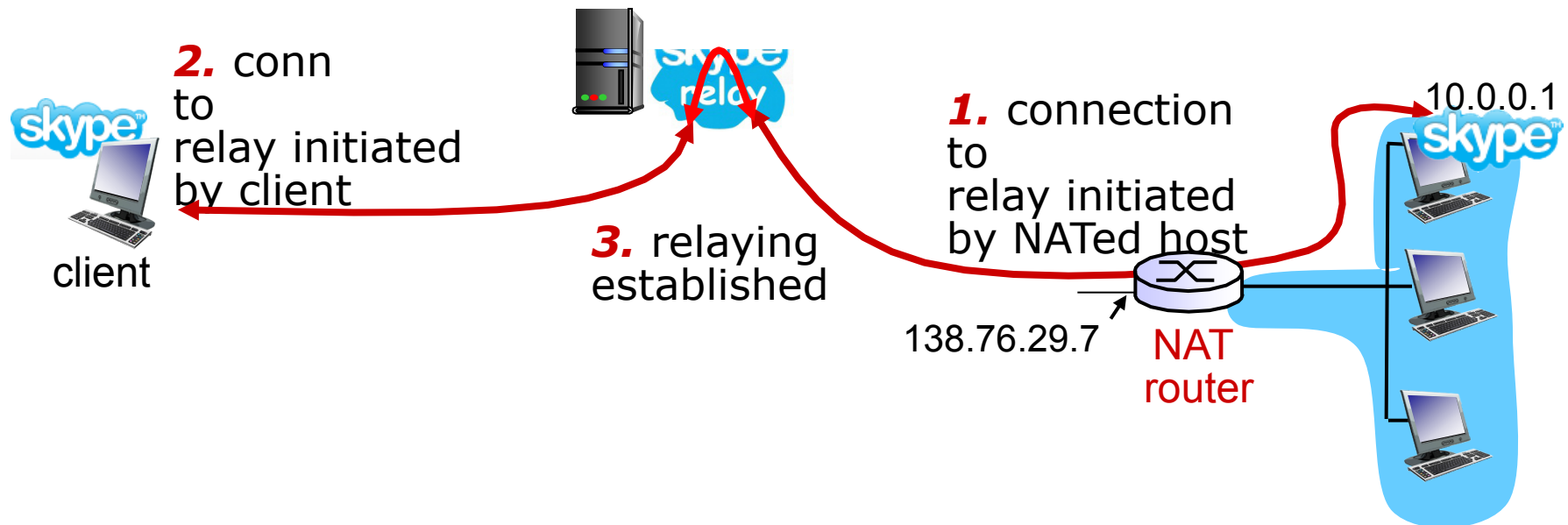
- ❖ address mappings (with lease times)

i.e., automate static NAT port map configuration



NAT traversal problem

- ❖ *solution 3:* relaying (used in Skype)
 - NATed client establishes connection to relay
 - external client connects to relay
 - relay bridges n to



Chapter 4: outline

4.1 introduction

4.2 virtual circuit and datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and multicast routing

ICMP: internet control message protocol

- ❖ used by hosts & routers to communicate network-level

<u>Type</u>	<u>Code</u>	<u>description</u>
0	0	echo reply (ping)
3	0	dest. network unreachable
		dest host unreachable

le

unre

network, port,
protocol

- echo request/reply (used by ping)

- ❖ network-layer "above" IP:

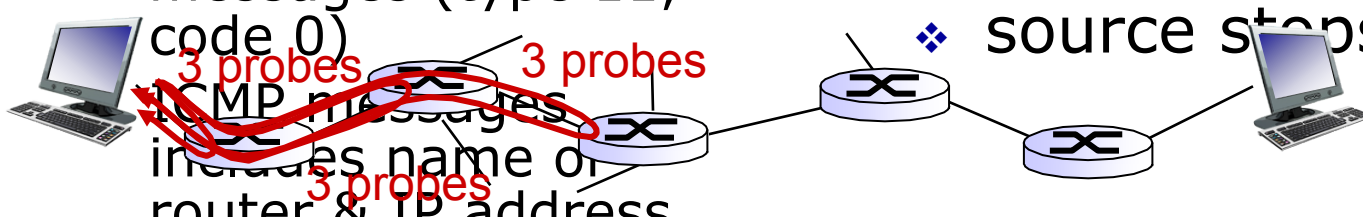
- ICMP msgs carried in IP datagrams

		unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

- ❖ **ICMP message:** type, code plus first 8

Traceroute and ICMP

- ❖ source sends series of UDP segments to dest
 - first set has TTL = 1
 - second set has
 - ❖ when ICMP messages arrives, source records RTTs
- ing criteria:*

- ❖ when datagrams arrives to nth router:
 - router discards datagrams
 - and sends source ICMP messages (type 11, code 0)
 - ❖ destination returns ICMP "port unreachable" message (type 3, code 3)
 - ❖ source stops
- 
- The diagram illustrates a network topology for a traceroute. It shows a sequence of four nodes connected in a line: a laptop on the left, followed by three routers (represented by circles with an 'X'), and a laptop on the right. Red annotations are present: '3 probes' is written in red above each of the three routers. Red arrows point from each router back to the leftmost laptop, with the text 'ICMP messages' written in red above the arrows. Below the first router, the text 'increases name of router & IP address' is written in red.

IPv6: motivation

- ❖ *initial motivation*: 32-bit address space soon to be completely allocated.
- ❖ additional motivation:

- hea

IPv6 datagram format:

- fixed-length 40 byte header
- no fragmentation allowed

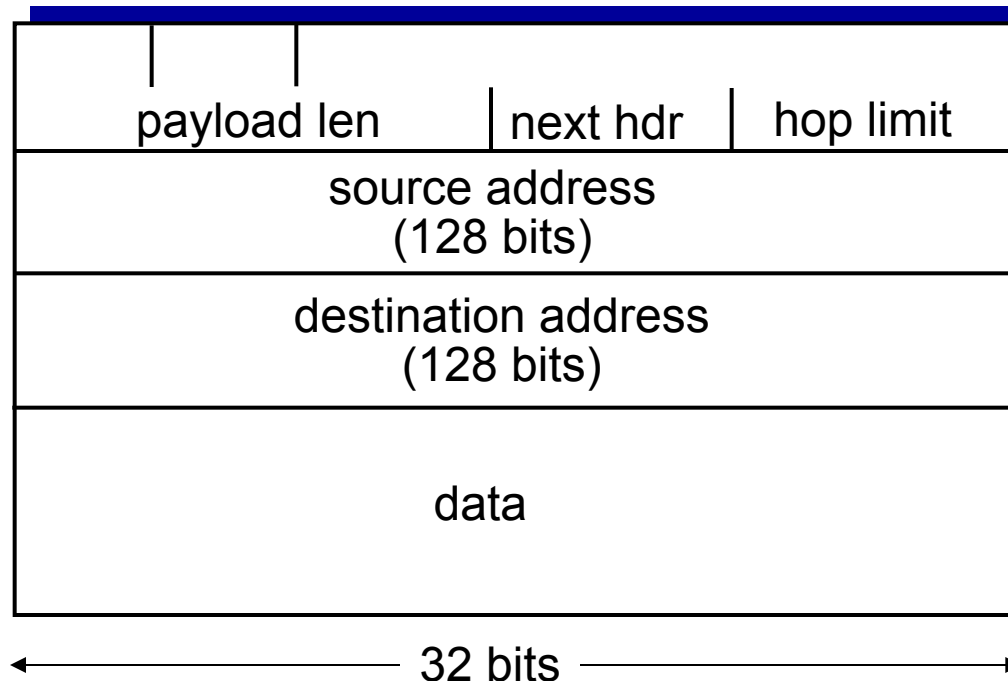
IPv6 datagram format

priority: identify priority among datagrams in flow

flow Label: identify datagrams in same "flow."

(concept of "flow" not well defined)

r da



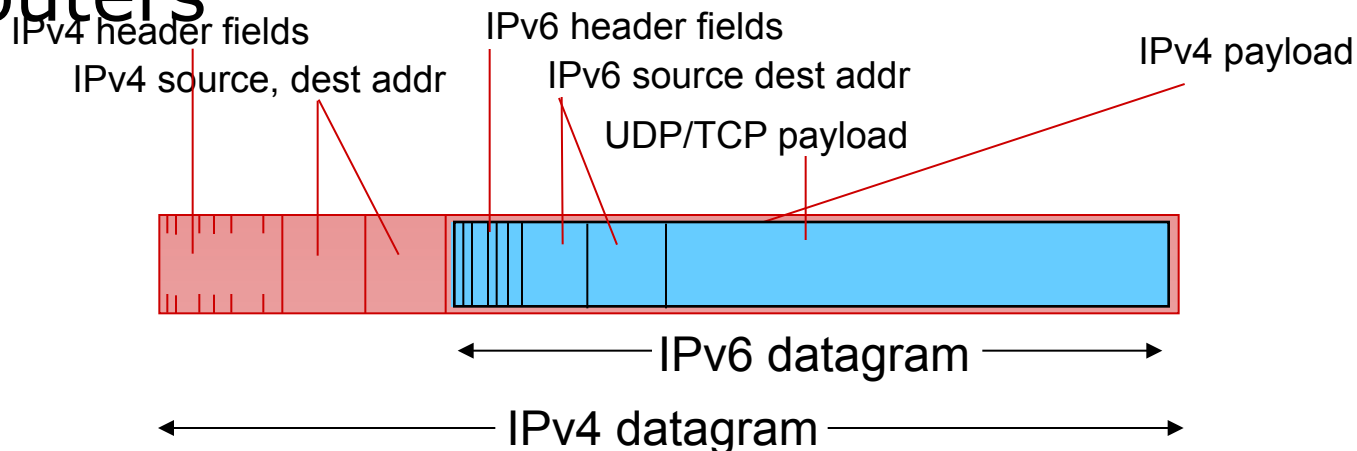
Other changes from IPv4

- ❖ *checksum*: removed entirely to reduce processing time at each hop
- ❖ *options*: allow extension of header,
- ❖
 - add "Big" Too
 - multicast group management functions

Transition from IPv4 to IPv6

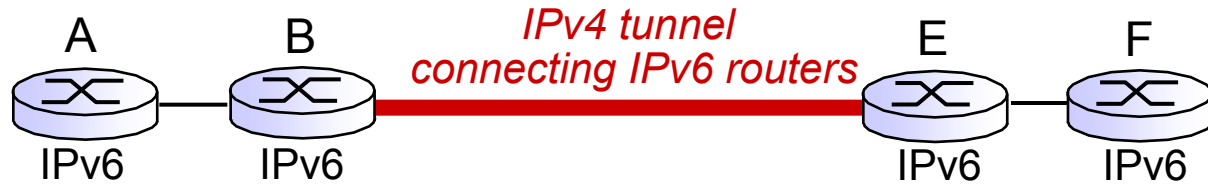
- ❖ not all routers can be upgraded simultaneously
 - no “flag days”

- ❖ *tunn* ^S
payload in IPv4 datagram among IPv4 routers

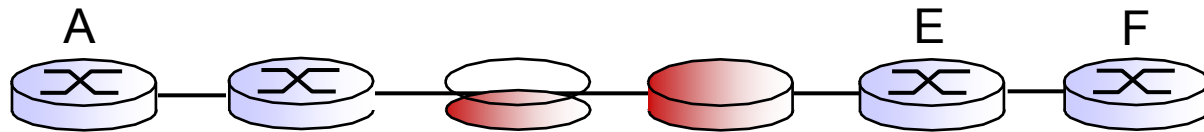


Tunneling

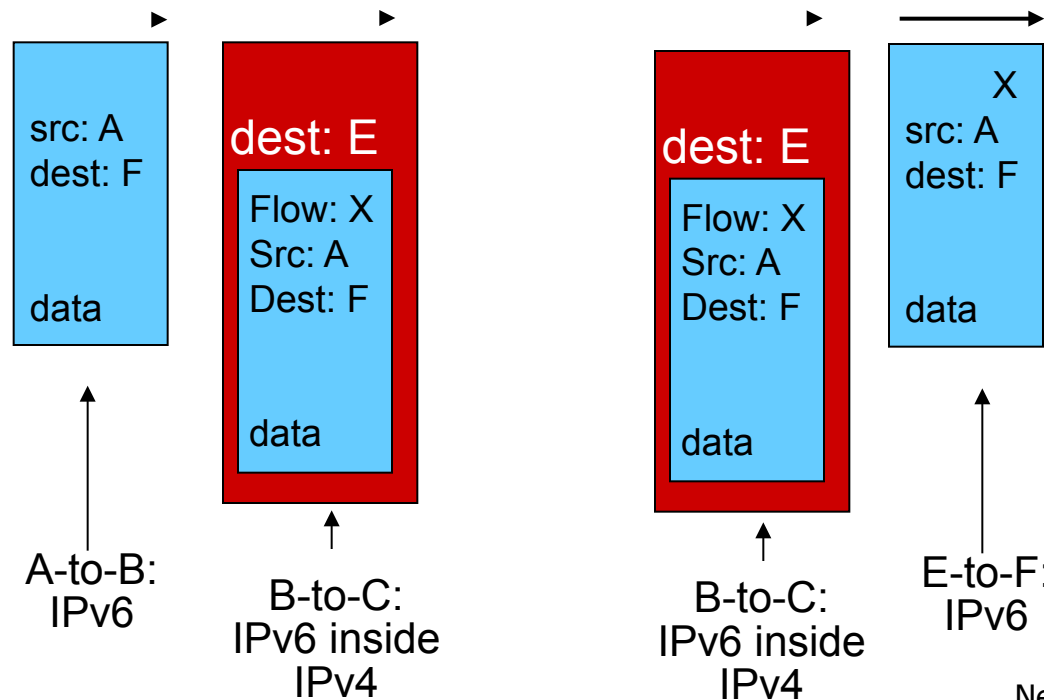
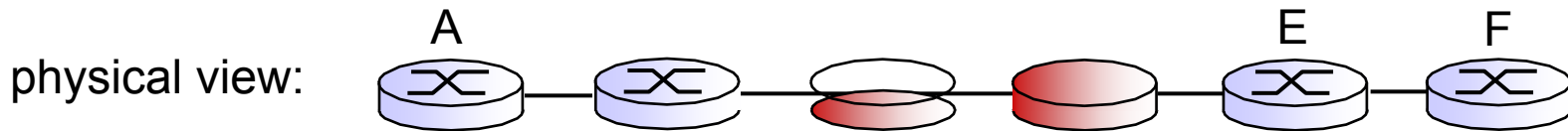
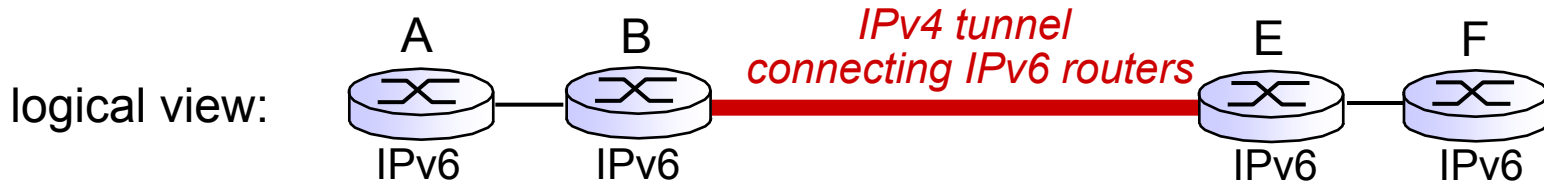
logical view:



physical view:



Tunneling



IPv6: ~~adoption~~

- ❖ US National Institutes of Standards estimate [2013]:
 - ~3% of industry IP routers
 - ~11% of US
- ❖ *Long* *se*
 - 20 years and counting!
 - think of application-level changes in last 20 years: WWW, Facebook, ...
 - *Why?*

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

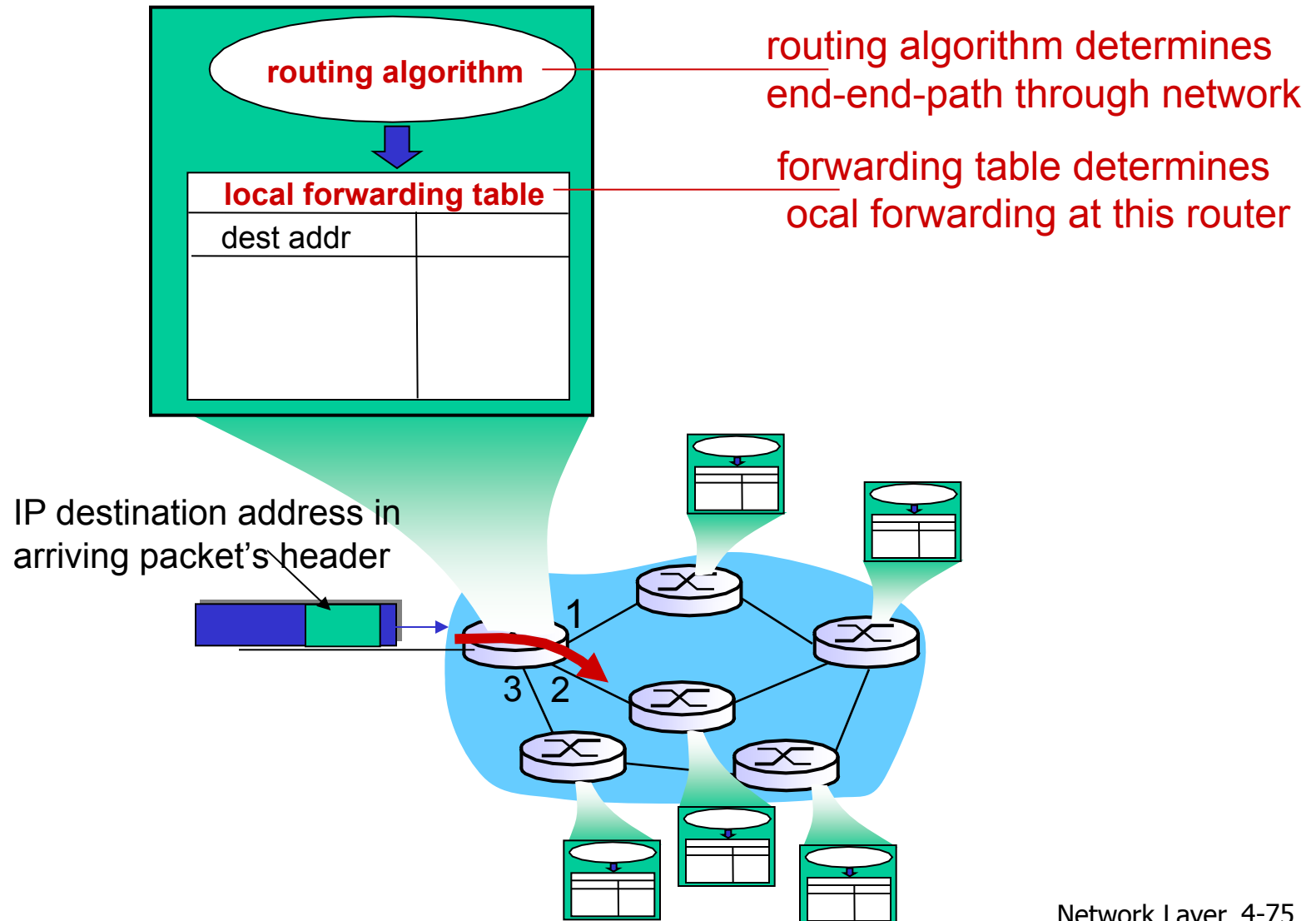
4.5 routing algorithms

- link state
- distance vector
archical routing

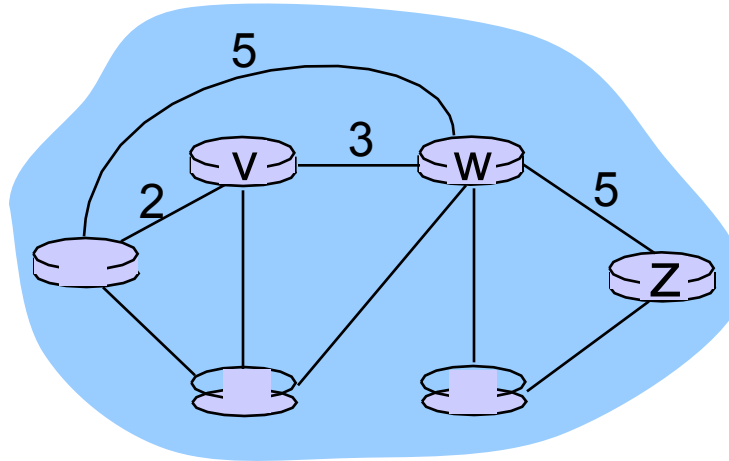
- OSPF
- BGP

4.7 broadcast and multicast routing

Interplay between routing, forwarding



Graph abstraction

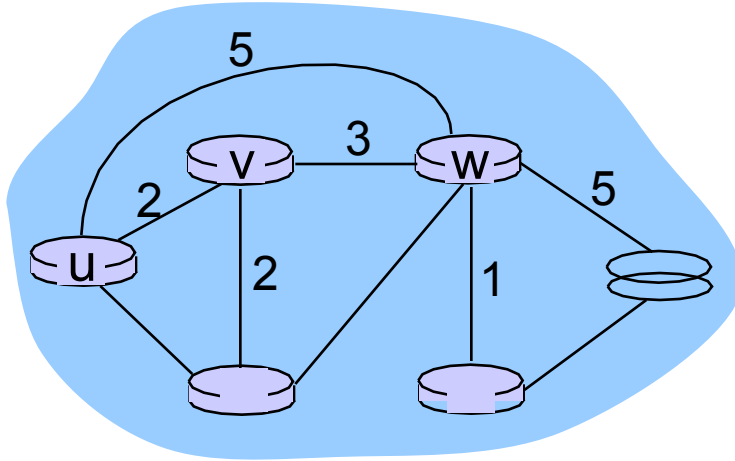


N = set of routers = $\{ u, v, w, x, y, z \}$

E = set of links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

aside: graph abstraction is useful in other network contexts, e.g., P2P, where N is set of peers and E is set of TCP connections

Graph abstraction: costs



$c(x, x') = \text{cost of link } (x, x')$

e.g., $c(w, z) = 5$

could always be 1, or
dth,

cost of path $(x_1, x_2, x_3, \dots, x_p) = c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$

key question: what is the least-cost path between u
routing algorithm: algorithm that finds that least c

Routing algorithm classification

Q: global or decentralized information?

global:

- ❖ all routers have

- ❖ “link s

decentralized:

- ❖ router knows physically-connected neighbors, link costs to neighbors

- ❖ iterative process of computation, exchange of info with neighbors

- ❖ “distance vector”

Q: static or dynamic?

static:

tes change slowly

- ❖ routes change more quickly
 - periodic update
 - in response to link cost changes

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and
datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and
multicast routing

A Link-State Routing Algorithm

Dijkstra's algorithm

- ❖ net topology, link costs known to nodes
 - all n info
- ❖ computes least cost paths from one node ('source') to all other nodes
 - gives *forwarding table* for that node
- ❖ iterative: after k iterations, know least cost path to k

notation:

- ❖ $c(x,y)$: link cost from x to y; $= \infty$ if not neighbors
- ❖ $p(v)$: predecessor node along path from source to v
- ❖ N' : set of nodes whose least cost path definitively known

Dijkstra's Algorithm

1 **Initialization:**

2 $N' = \{u\}$

3 for all nodes v

4 if v adjacent t

7

8 **Lo**

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min(D(v), D(w) + c(w,v))$**

13 /* new cost to v is either old cost to v or known

14 shortest path cost to w plus cost from w to v */

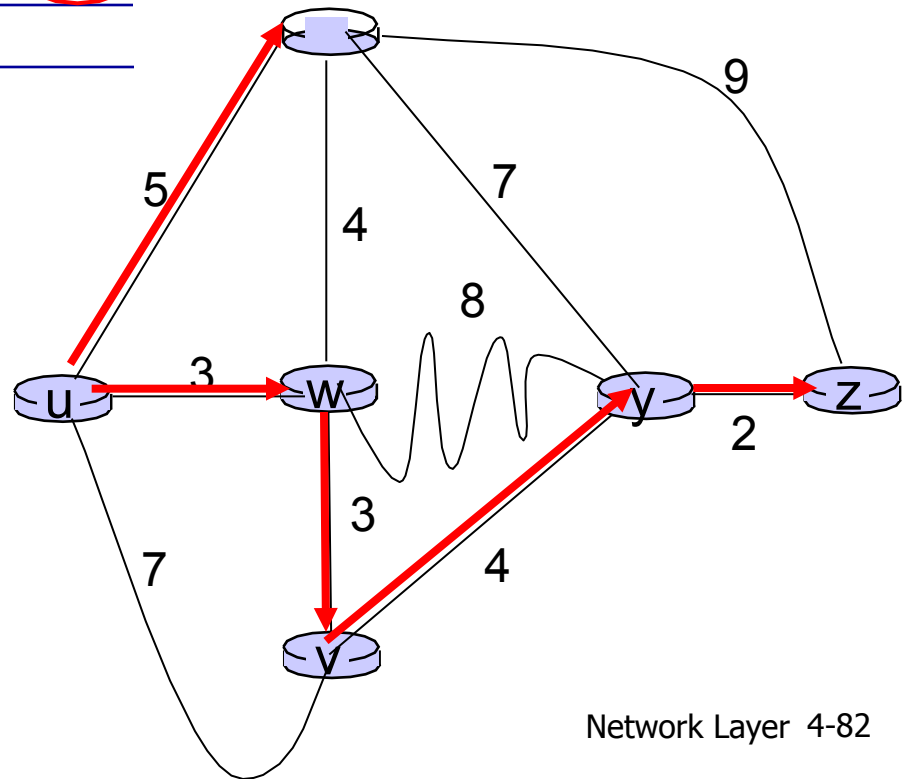
15 **until all nodes in N'**

Dijkstra's algorithm: example

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv					
4						
5						

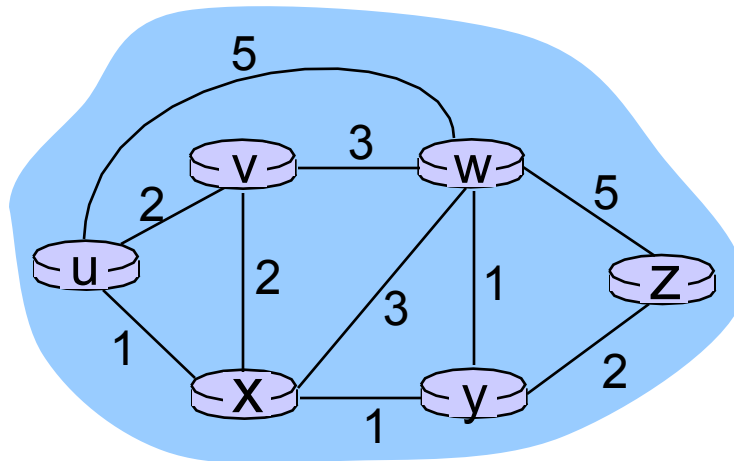
notes:

- ❖ construct shortest path tree by tracing predecessor nodes
- ❖ ties can exist (can be broken arbitrarily)



Dijkstra's algorithm: another example

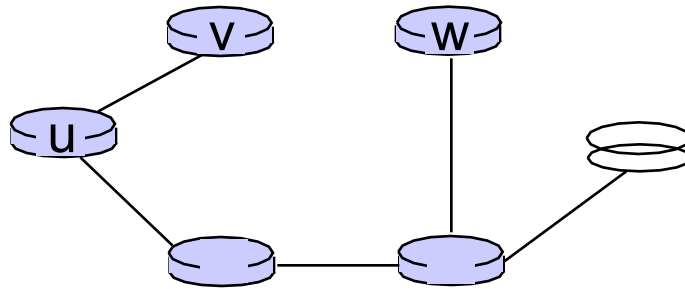
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2				4,y
						,y
						,y
5	uxyv					



Dijkstra's algorithm: example

(2)

resulting shortest-path tree from u:



resulting forwarding table in u:

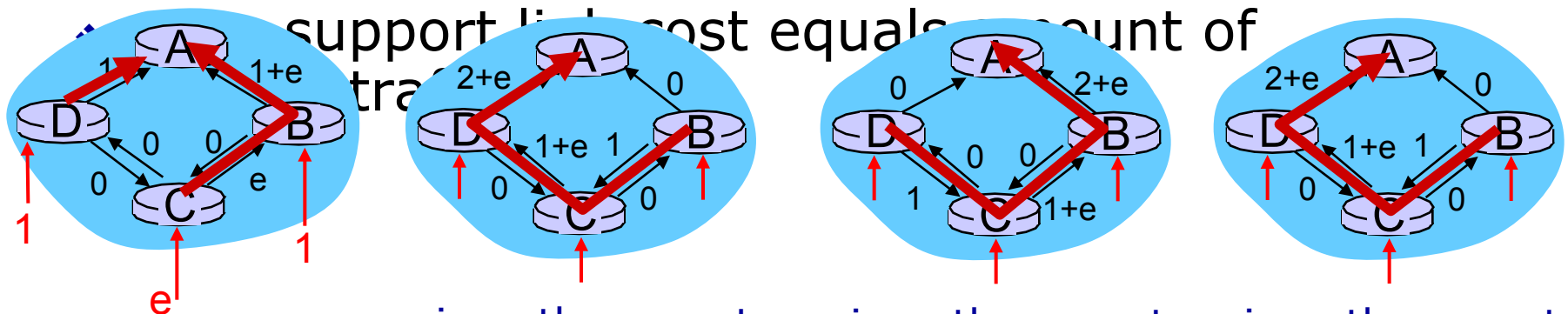
destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

Dijkstra's algorithm, discussion

algorithm complexity: n nodes

- ❖ each iteration: need to check all nodes, w, not in N
- ❖ $n(n+1)/2$ com

oscilla



given these costs, find new routing.... resulting in new costs
 given these costs, find new routing.... resulting in new costs
 given these costs, find new routing.... resulting in new costs
 given these costs, find new routing.... resulting in new costs

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and
datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and
multicast routing

Distance vector algorithm

Bellman-Ford equation (dynamic programming)

|

y^x

then

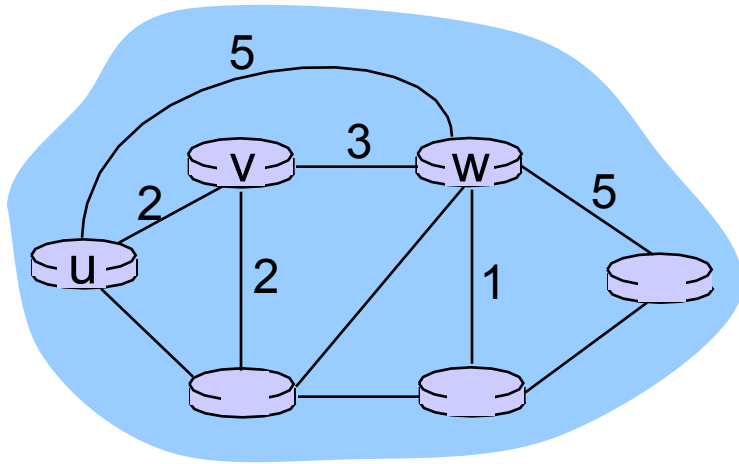
$$d_x(y) = \min_v \{ c(x, v) + d_v(y) \}$$

cost from neighbor v to destination y

cost to neighbor v

min taken over all neighbors v of x

Bellman-Ford example



clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

says:

$$\begin{aligned} & c(u,w) + d_w(z), \\ & = \min \{ 2 + 5, \\ & \quad 1 + 3, \\ & \quad 5 + 3 \} = 4 \end{aligned}$$

node achieving minimum is next
hop in shortest path, used in forwarding table

Distance vector algorithm

❖ $D_x(y)$ = estimate of least cost from x to y

- x maintains

$$\mathbf{D}_x = [D_x(y) : y$$

- kn (x, v)

- maintains its neighbors' distance vectors. For each neighbor v , x maintains

$$\mathbf{D}_v = [D_v(y) : y \in N]$$

Distance vector algorithm

key idea:

❖ from time-to-time, each node sends its own distance to

❖
neig
B-F equation: $D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\}$ for each node $y \in N$ ng

❖ under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

Distance vector algorithm

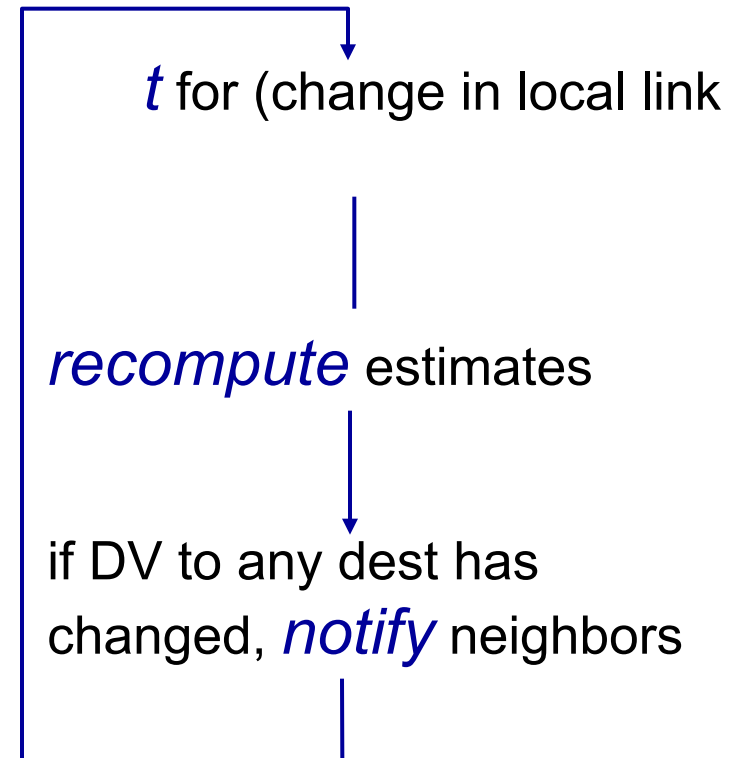
iterative,
asynchronous:
each local iteration
caused by:

- ❖ DV u
from neighbor

distributed:

- ❖ each node notifies
neighbors *only* when
its DV changes
 - neighbors then notify
their neighbors if
necessary

each node:



$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

**node x
table**

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

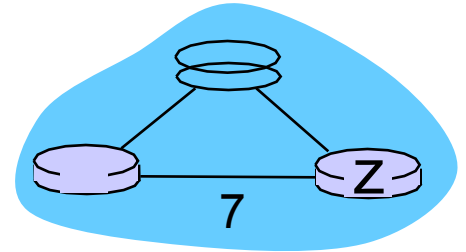
**no
t**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

**node z
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

time



$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

node x
table

	cost to			
	x	y	z	
from x	0	2	7	
from y	∞	∞	∞	
from z	∞	∞	∞	

no
t

	x	y	z	
from x	∞	∞		
from y	2	0	1	
from z	∞	∞	∞	

node z
table

	cost to			
	x	y	z	
from x	∞	∞	∞	
from y	∞	∞	∞	
from z	7	1	0	

	cost to			
	x	y	z	
from x	0	2	3	
from y	2	0	1	
from z	7	1	0	

	x	y	z	
from x	∞	∞		
from y	2	0	1	
from z	7	1	0	

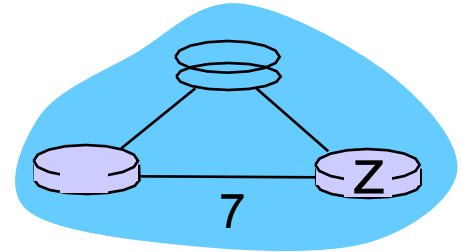
	cost to			
	x	y	z	
from x	0	2	7	
from y	2	0	1	
from z	3	1	0	

	cost to			
	x	y	z	
from x	0	2	3	
from y	2	0	1	
from z		1	0	

	x	y	z	
from x	∞	∞		
from y	2	0	1	
from z	3	1	0	

	cost to			
	x	y	z	
from x	0	2	3	
from y	2	0	1	
from z	3	1	0	

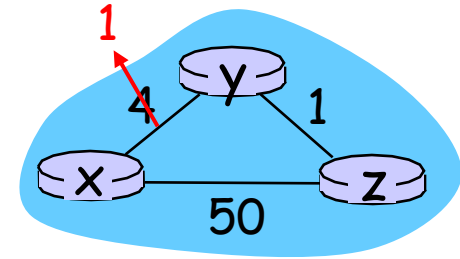
time



Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ updates routing



“good news travels fast”

if DV neighbors

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

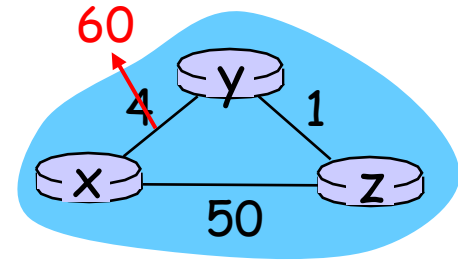
t_1 : z receives update from y, updates its table, computes new least cost to x, sends its neighbors its DV.

t_2 : y receives z's update, updates its distance table. y's least costs do *not* change, so y does *not* send a message to z.

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ *bad news travel*



- ❖ algorithm

poisoned reverse:

- ❖ If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❖ will this completely solve count to infinity problem?

Comparison of LS and DV algorithms

message complexity

- ❖ **LS:** with n nodes, E links, $O(nE)$ msgs sent
- ❖ **DV:** exchange between neighbors only

robustness: what happens if router malfunctions?

speed convergence

- ❖ **LS:** $O(n^2)$ algorithm requires $O(nE)$ msgs
 - may have oscillations
- ❖ **DV:** convergence time varies
 - may be routing loops
 - count-to-infinity problem

able

DV:

- DV node can advertise incorrect *path* cost
- each node's table used by others
 - error propagate thru network

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and
datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
 - distance vector
- archical routing

- OSPF
- BGP

4.7 broadcast and
multicast routing

Hierarchical ~~routing~~

our routing study thus far -
idealization

- ❖ all routers identical
- ❖ network

scale:

million

destinations:

- ❖ can't store all dest's
in routing tables!
- ❖ routing table
exchange would
swamp links!

autonomy

- ❖ internet = network of
networks
- ❖ each network admin
may want to control
routing in its own
network

Hierarchical routing

- ❖ aggregate routers into regions, “autonomous systems” (AS

gateway router:

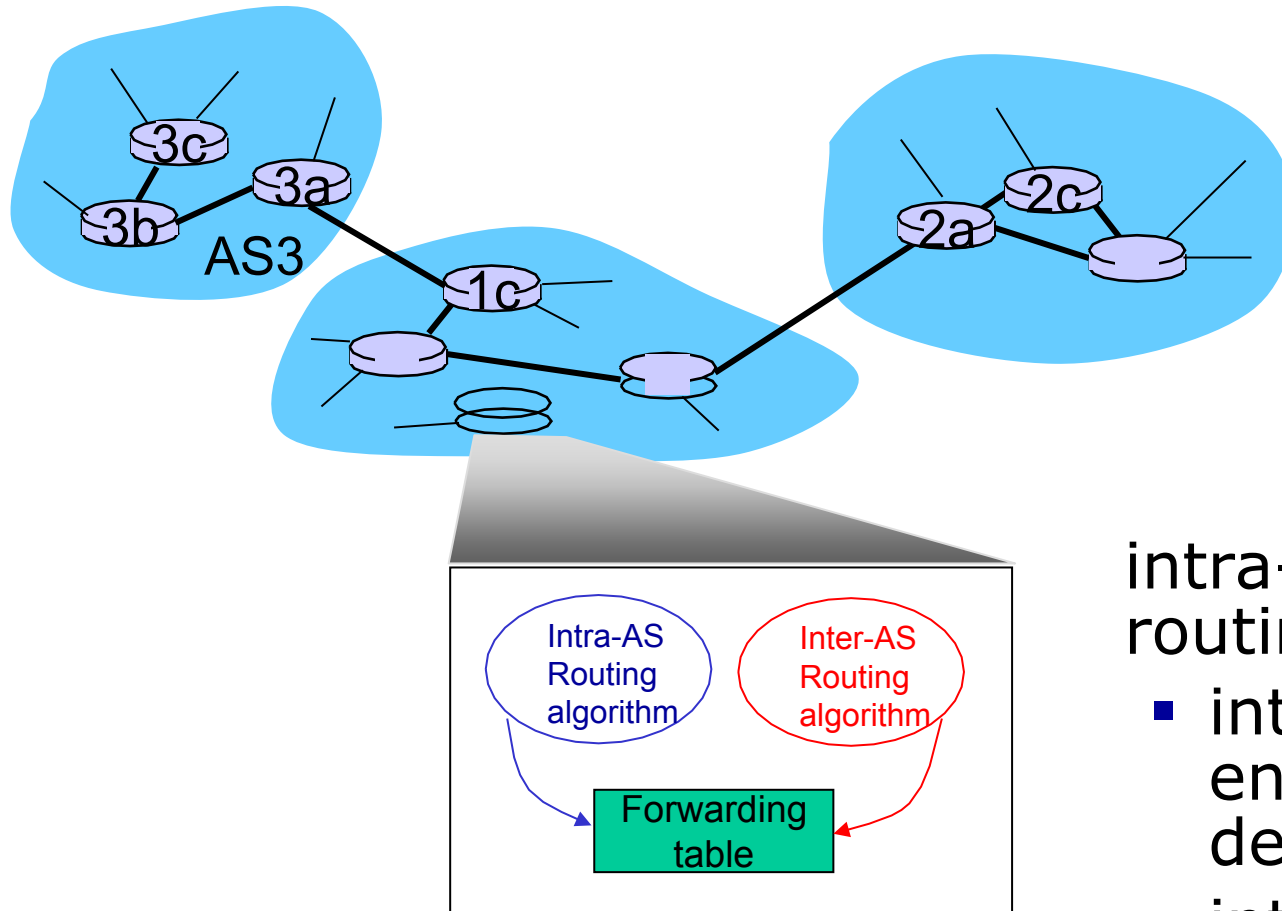
- ❖ at “edge” of its own AS

link to router in

AS r
routing protocol

- “intra-AS” routing protocol
- routers in different AS can run different intra-AS routing protocol

Interconnected ASes



- by both
intra- and inter-AS
routing algorithm
- intra-AS sets entries for internal dests
 - inter-AS & intra-AS sets entries for external dests

Inter-AS tasks

- ❖ suppose router in AS1 receives datagram destined outside of AS1:
 - router should

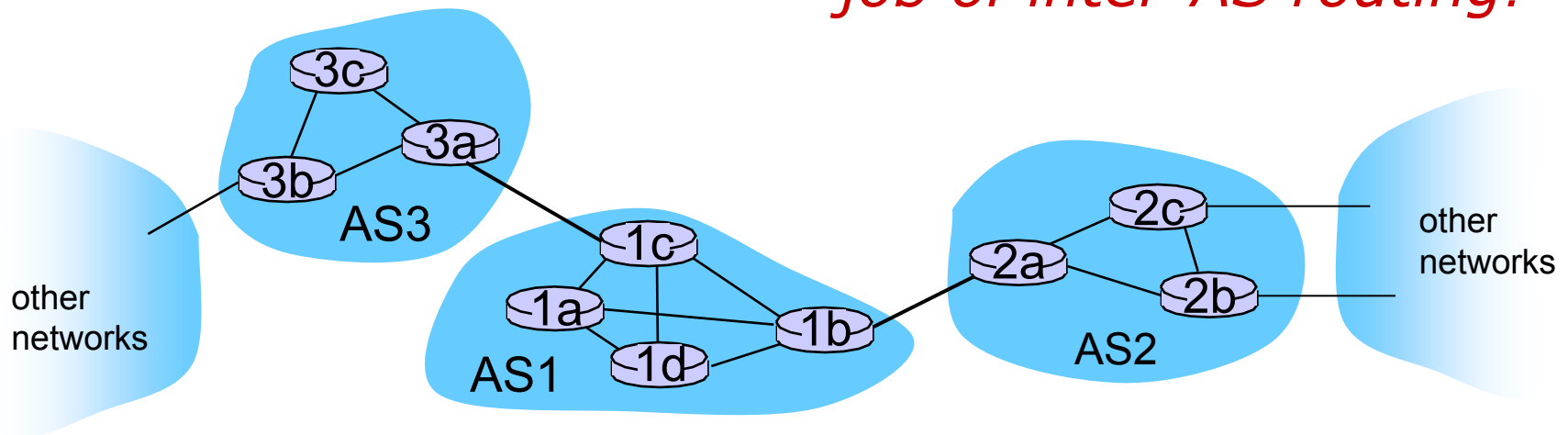
AS1 must:

1. learn which destinations are reachable through AS2, which through AS3

but

AS1

job of inter-AS routing!

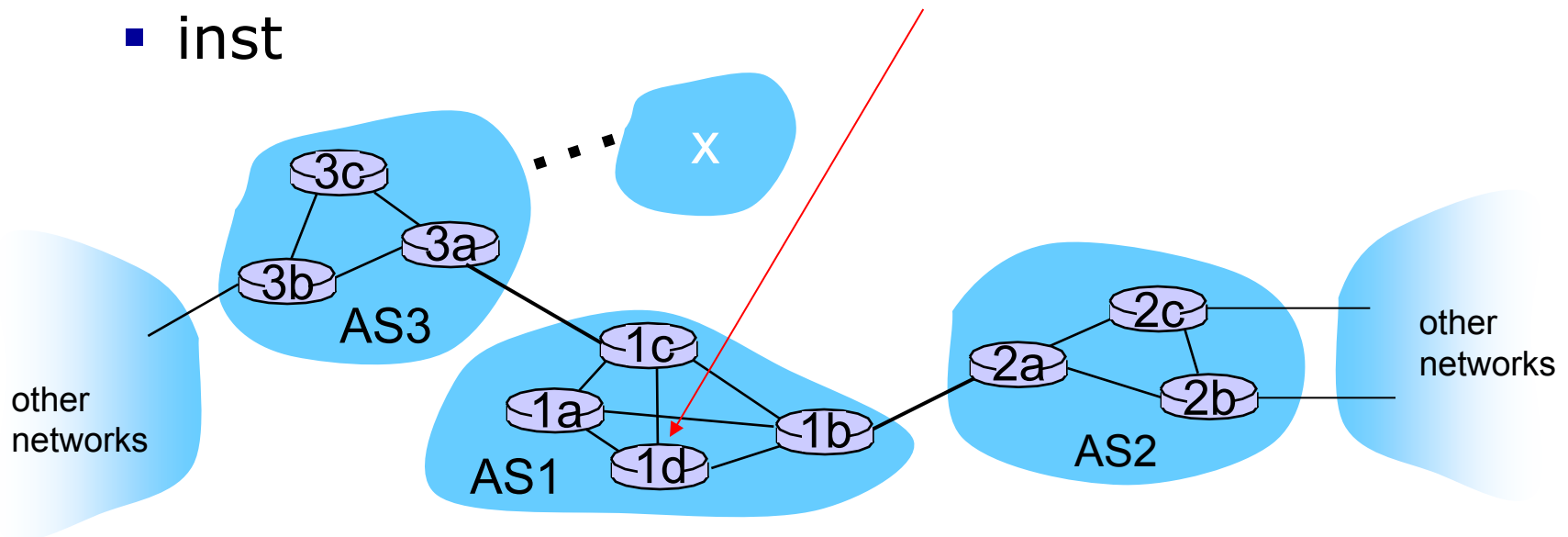


Example: setting forwarding table in ~~router 1d~~

- ❖ suppose AS1 learns (via inter-AS protocol) that subnet **x** reachable via AS3 (gateway 1c), but not via AS2
 - inter-AS protocol propagates reachability info to all internal ro

❖

- inst

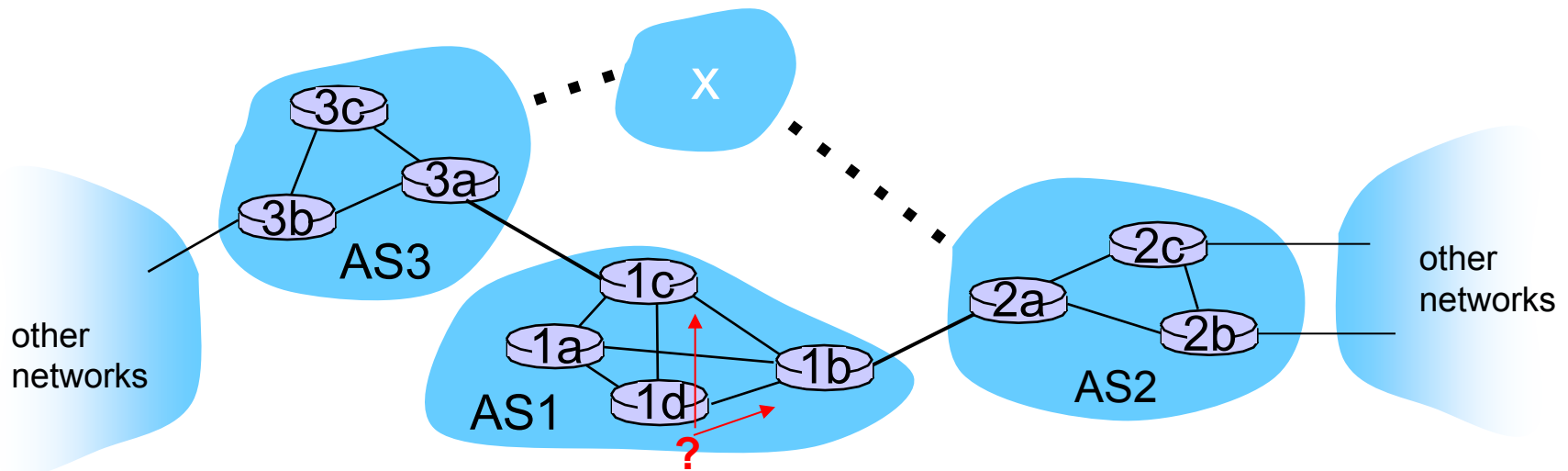


Example: choosing among multiple ~~ASes~~

- ❖ now suppose AS1 learns from inter-AS protocol that subnet **x** is reachable from AS3 *and* from AS2.
- ❖ to configure for user 1d must

■ this

col!

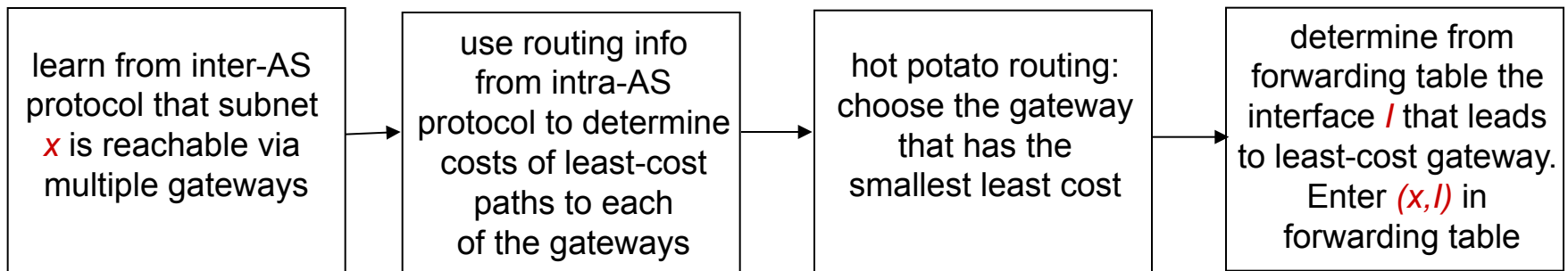


Example: choosing among multiple ~~ASes~~

- ❖ now suppose AS1 learns from inter-AS protocol that subnet **x** is reachable from AS3 *and* from AS2.
- ❖ to configure forwarding table, router 1d must determine toward which it should

- ❖ *hot potato* of two routers.

losest



Chapter 4: outline

4.1 introduction

4.2 virtual circuit and datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and multicast routing

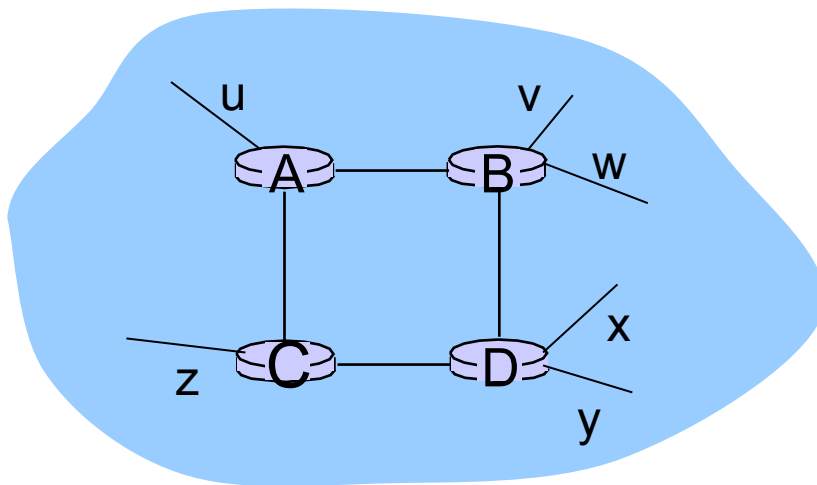
Intra-AS Routing

- ❖ also known as *interior gateway protocols (IGP)*
- ❖ most common routing
 - OSPF: Open Shortest Path First
 - IGRP: Interior Gateway Routing Protocol (Cisco proprietary)

RIP (Routing Information Protocol)

- ❖ included in BSD-UNIX distribution in 1982
- ❖ distance vector algorithm
 - distance metric: # hops (max = 15 hops), each link has cost 1
 - DVs exchanged 30 sec in response

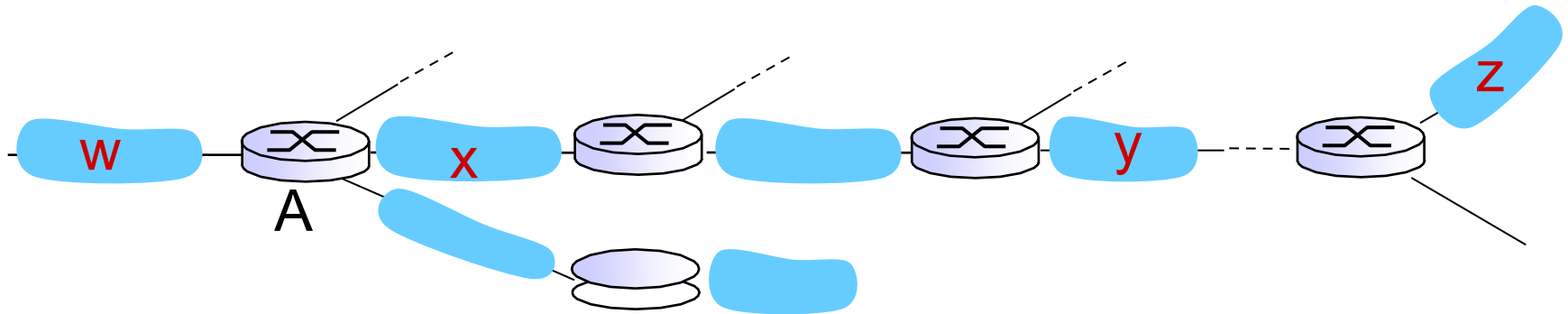
(in IP



from router A to destination *subnets*:

<u>subnet</u>	<u>hops</u>
u	1
v	2
w	2
x	3
y	3
z	2

RIP: example



routing table in router D

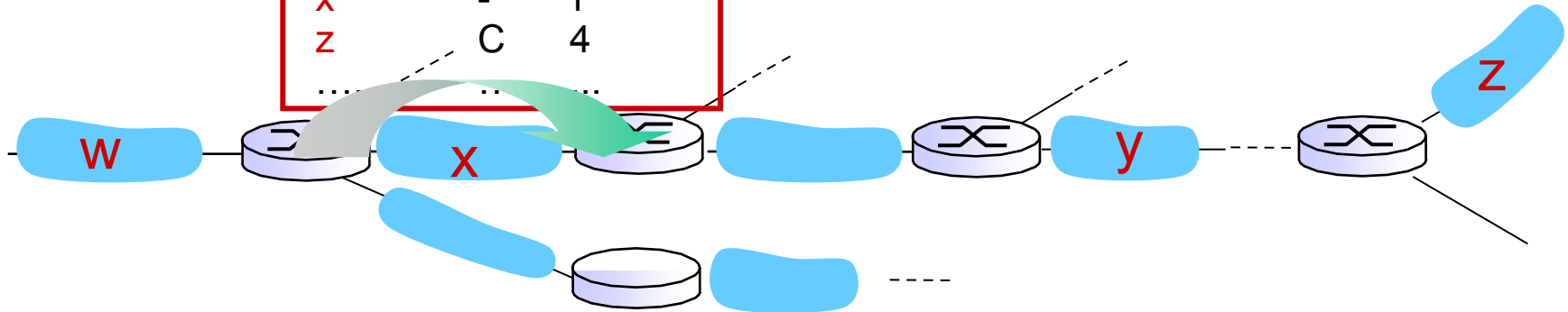
destination subnet	next router	# hops to dest
w	A	2
y	B	2
z	B	7
x	--	1
....		

RIP:

example

A-to-D advertisement

dest	next	hops
W	-	1
X	-	1
Z	C	4
...	..	...



routing table in router D

destination subnet	next router	# hops to dest
W	A	2
y	B	2
Z	B A	7 5
X	--	1
....		

RIP: link failure, recovery

if no advertisement heard after 180 sec -->
neighbor/link declared dead

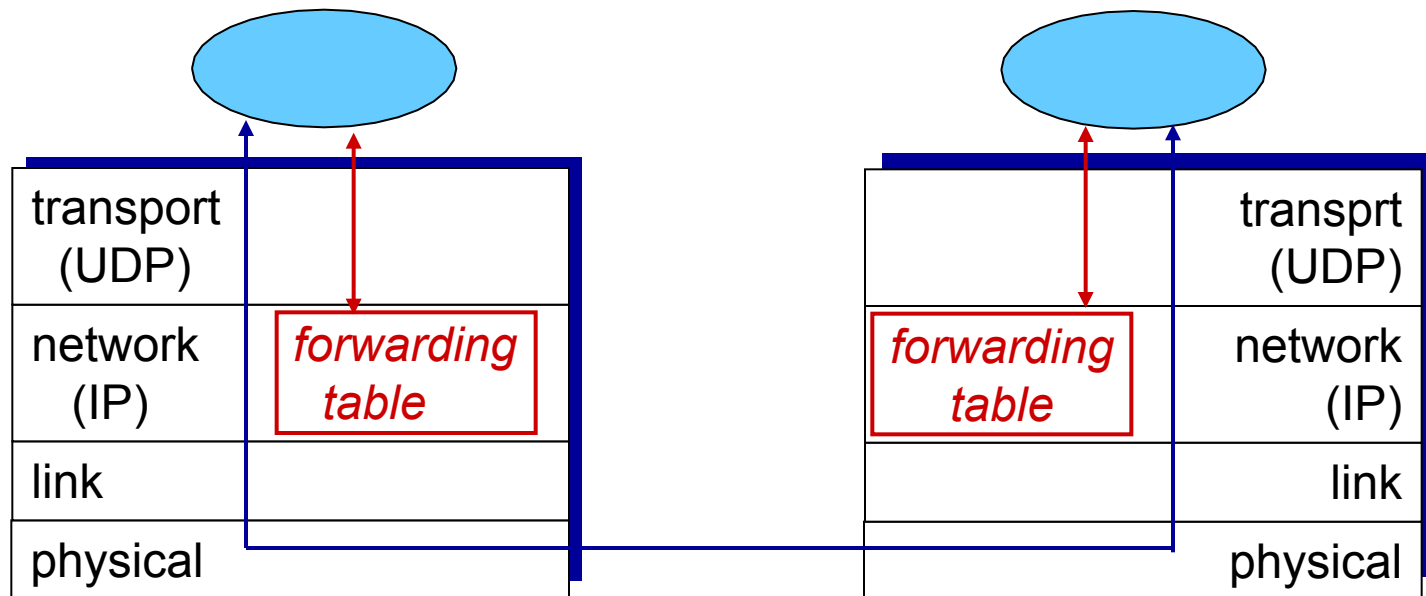
- routes via neighbor invalidated
- new advertisements from neighbors

S

- link failure advertised to entire net
- *poison reverse* used to prevent ping-pong loops (infinite distance = 16 hops)

RIP table processing

- ❖ RIP routing tables managed by *application-level* process called route-d (daemon)



OSPF (Open Shortest Path First)

- ❖ “open”: publicly available
- ❖ uses link state algorithm
 - LS packet di
- ❖ OSPF try per neighbor
- ❖ advertisements flooded to *entire* AS
 - carried in OSPF messages directly over IP (rather than TCP or UDP)
- ❖ *IS-IS routing* protocol: nearly identical to OSPF

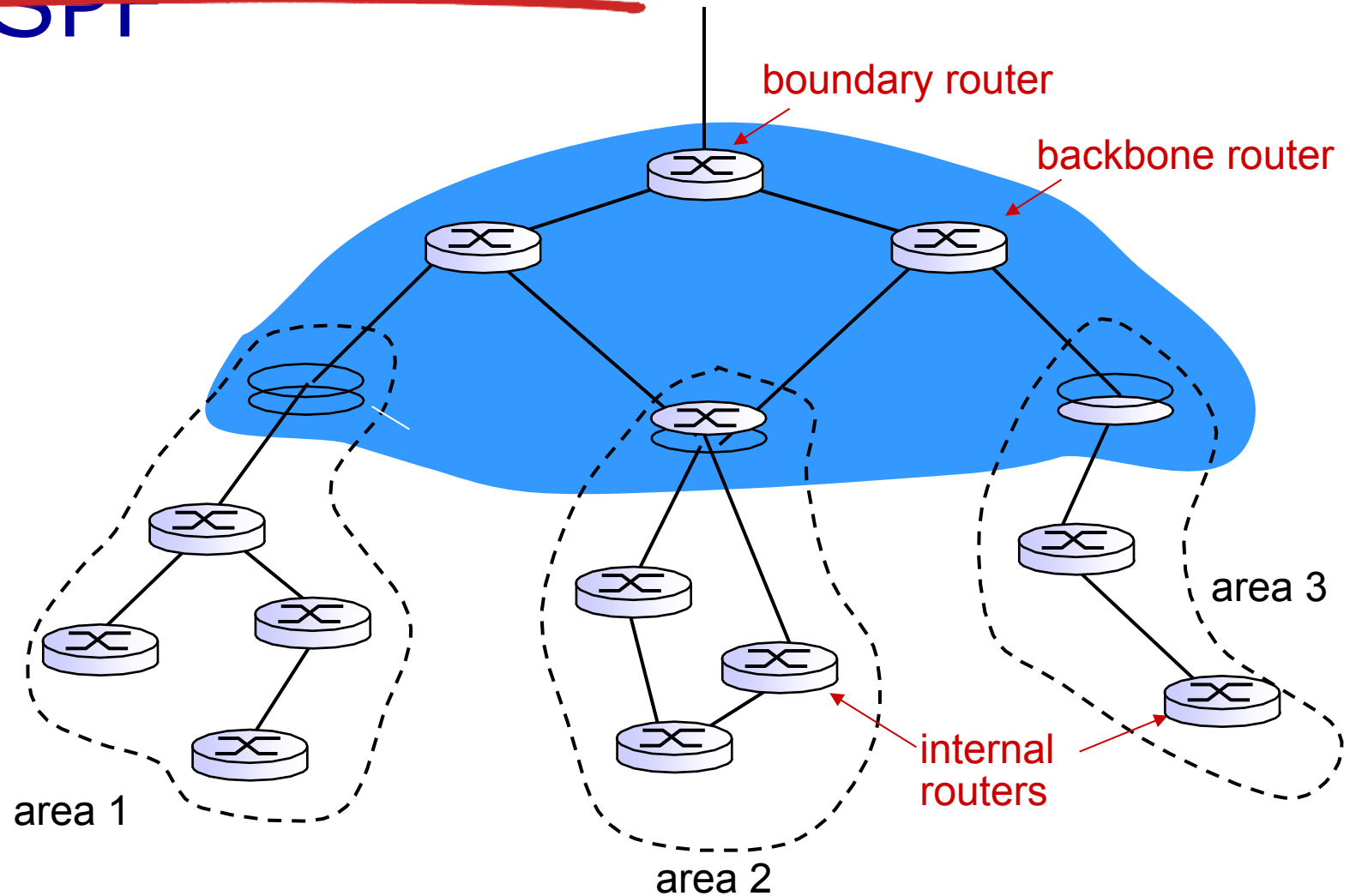
OSPF “advanced” features (not in RIP)

- ❖ *security*: all OSPF messages authenticated (to prevent malicious intrusion)
- ❖ **multiple** same-cost **paths** allowed (only one path in R

differ t set
“low” for best effort ToS; high for real time ToS)

- ❖ integrated uni- and **multicast** support:
 - Multicast OSPF (MOSPF) uses same topology data base as OSPF
- ❖ **hierarchical** OSPF in large domains.

Hierarchical ~~OSPF~~



Hierarchical ~~OSPF~~

- ❖ *two-level hierarchy*: local area, backbone.
 - link-state advertisements only in area
 - each nodes area topology;
- ❖ *area*
distances to nets in own area, advertise to other Area Border routers.
- ❖ *backbone routers*: run OSPF routing limited to backbone.
- ❖ *boundary routers*: connect to other AS's.

Internet inter-AS routing:

~~BGP~~

- ❖ **BGP (Border Gateway Protocol):** *the* de facto inter-domain routing protocol
 - “glue that holds the Internet together”

inf

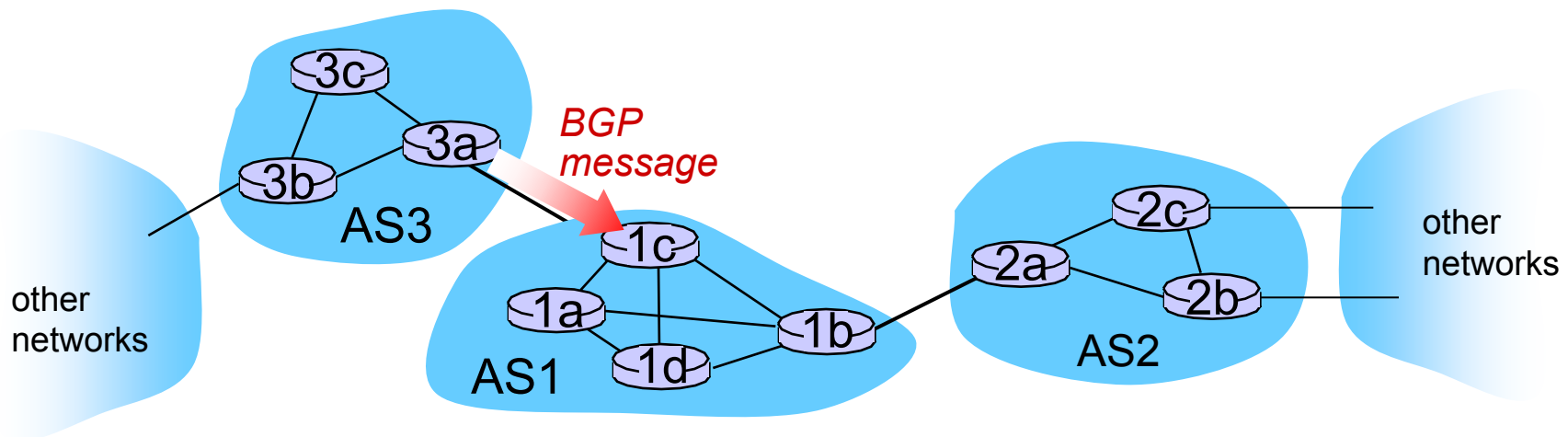
- **iBGP:** propagate reachability information to all AS-internal routers.
 - determine “good” routes to other networks based on reachability information and policy.
- ❖ allows subnet to advertise its existence to rest of Internet: *“I am here”*

BGP basics

- ❖ **BGP session:** two BGP routers (“peers”) exchange BGP messages:
 - advertising *paths* to different destination network prefixes (“path vector” protocol)
 - exchanged over semi-permanent TCP connections

- AS3

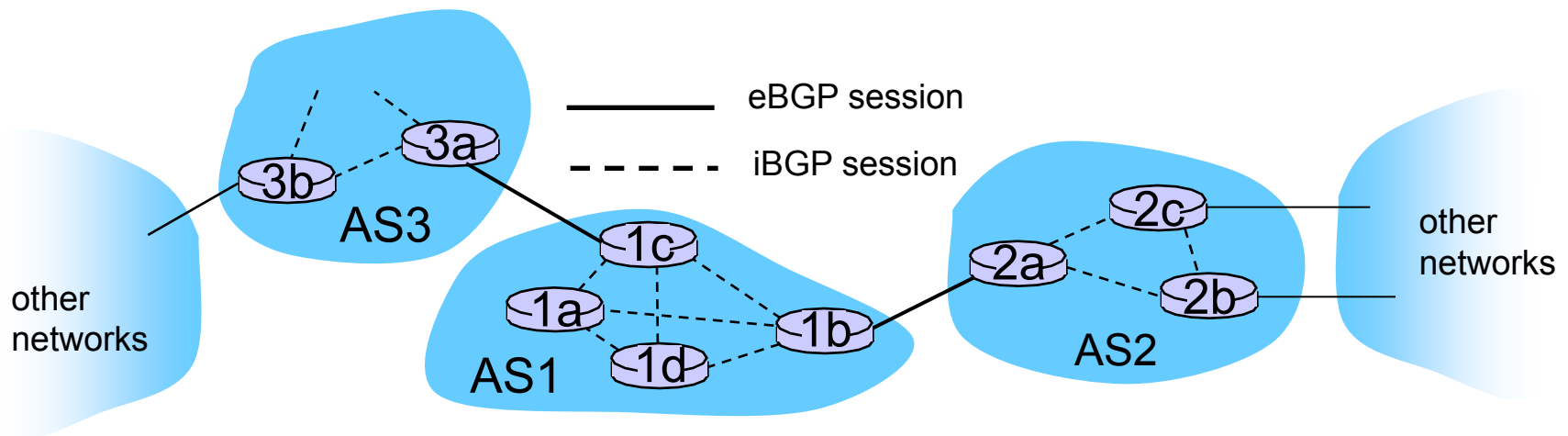
nt
efix



BGP basics: distributing path information

- ❖ using eBGP session between 3a and 1c, AS3 sends prefix reachability info to AS1.
 - 1c can then use iBGP to distribute new prefix info to all routers in AS1
 - 1b can then re-ability info to AS2

entry



Path attributes and BGP routes

- ❖ advertised prefix includes BGP attributes
 - prefix + attributes = “route”
- ❖ two important
 - **NEX** S router to next-hop AS. (may be multiple links from current AS to next-hop-AS)
- ❖ gateway router receiving route advertisement uses **import policy** to accept/decline
 - e.g., never route through AS x
 - ***policy-based*** routing

BGP route selection

- ❖ router may learn about more than 1 route to destination AS, selects route based on:
 1. local preference
 2. AS path length
 3. closest NEXT-HOP router: hot potato routing
 4. additional criteria

BGP messages

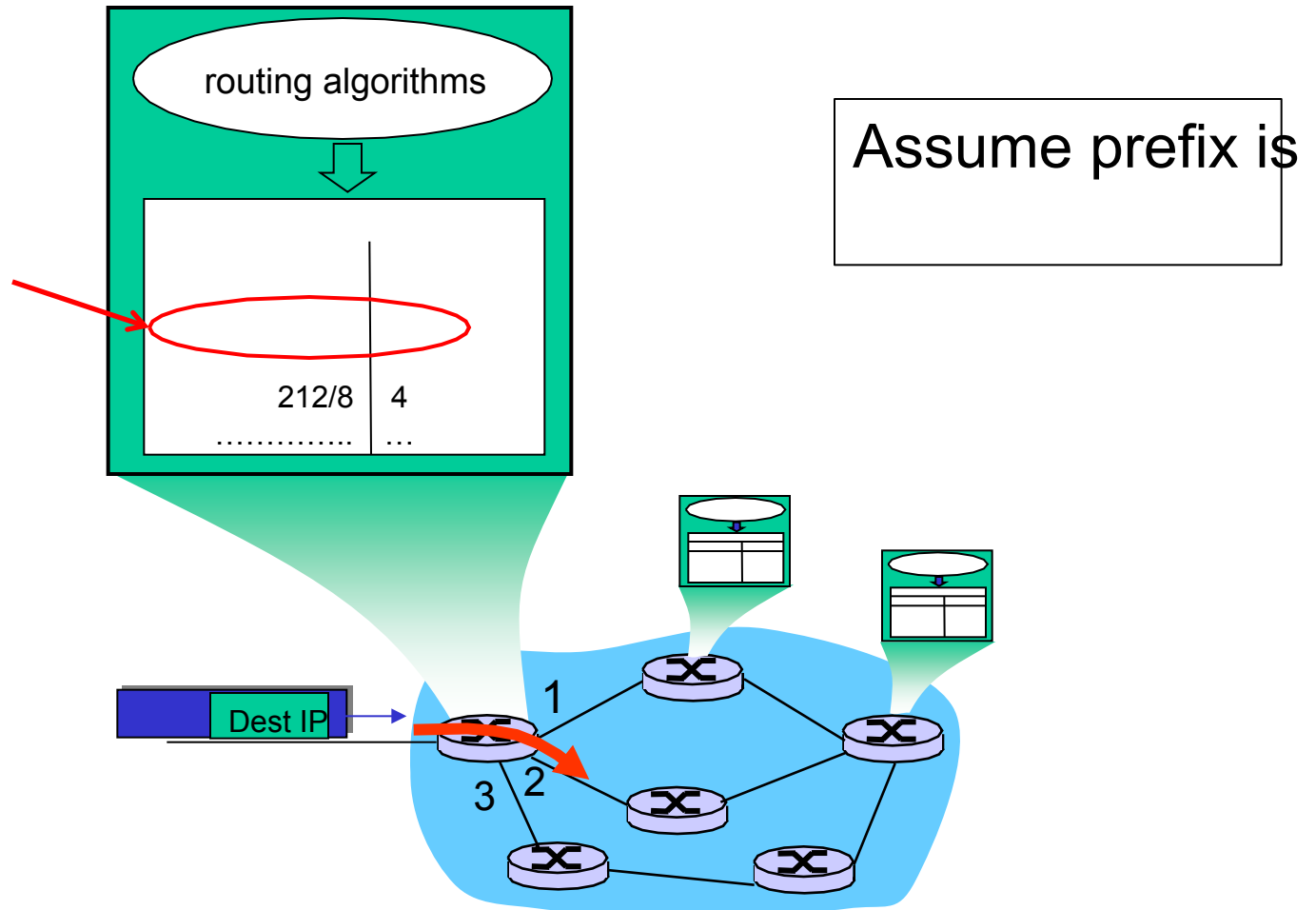
- ❖ BGP messages exchanged between peers over TCP connection
- ❖ BGP messages:
 - **OPEN:** opens connection to peer and exchanges capabilities
 - **KEEPALIVE:** keeps connection alive in absence of UPDATES; also ACKs OPEN request
 - **NOTIFICATION:** reports errors in previous msg; also used to close connection

Putting it Altogether:

How Does an Entry Get Into a Router's Forwarding Table?

- ❖ Ties together hierarchical routing (Section 4.5.3) with BGP (4.6.3) and OSPF (4.6.2).
- ❖ Provides nice overview of BGP!

How does entry get in forwarding table?



How does entry get in forwarding table?

High-level overview

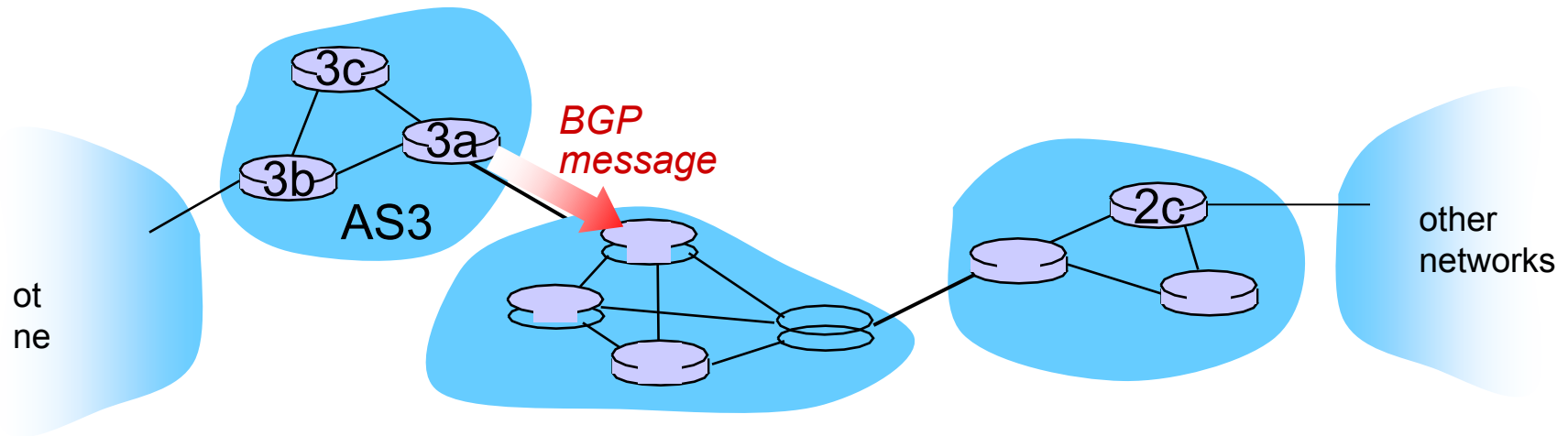
1. Router beco

prefix

3. Rou
table

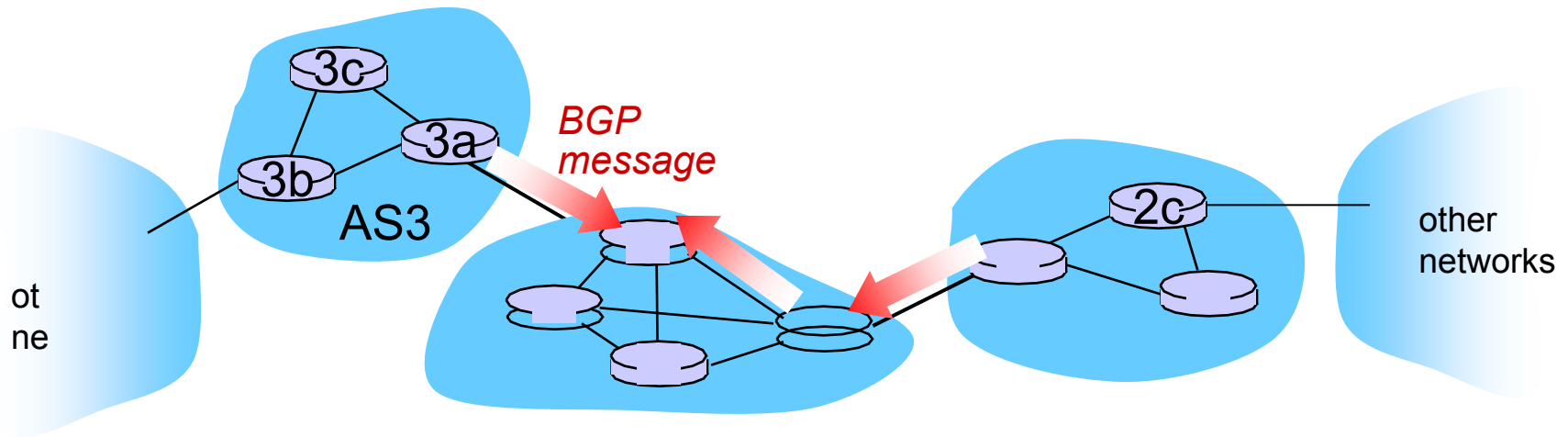
ding

Router becomes aware of ~~prefix~~



- ❖ BGP message contains “routes”
- ❖ “route” is a prefix and attributes: AS-PATH, NEXT-HOP,...
- ❖ Example: route:
 - ❖ Prefix: 138.16.64/22 ; AS-PATH: AS3 AS131 ; NEXT-HOP: 201.44.13.125

Router may receive multiple routes



- ❖ Router may receive multiple routes for same prefix
- ❖ Has to select one route

Select best BGP route to prefix

- ❖ Router selects route based on shortest AS-PATH

- ❖ AS

- ❖ AS3 AS131 AS201 to 138.16.64/22

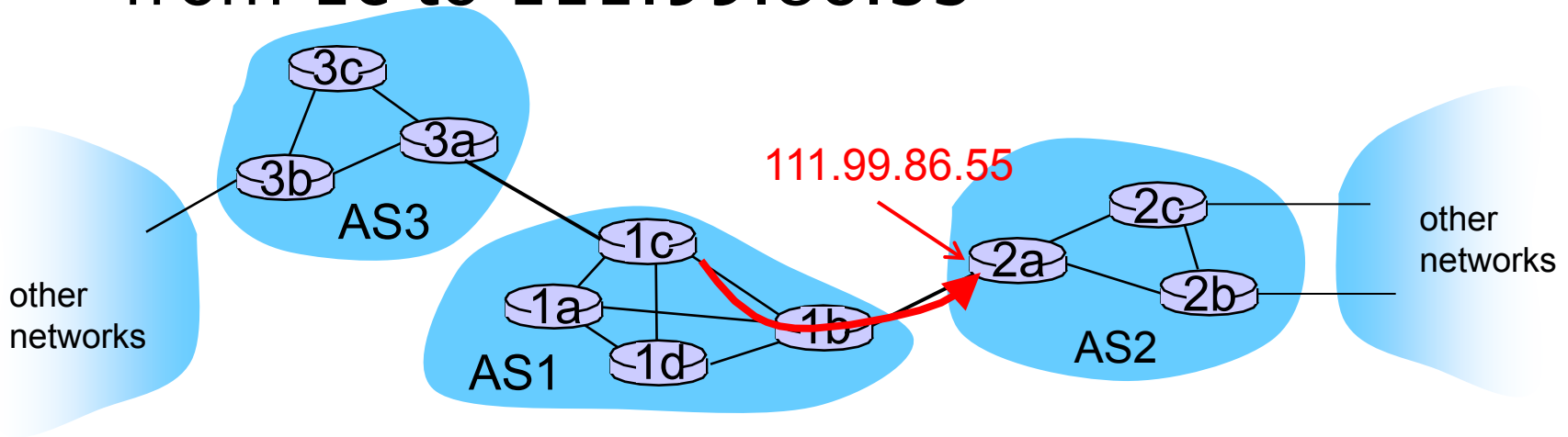
- ❖ What if there is a tie? We'll come back to that!

Find best intra-route to BGP route

- ❖ Use selected route's NEXT-HOP attribute
 - Route's NEXT-HOP attribute is the IP address of the router interface that begins the AS PATH.
- ❖ Example:

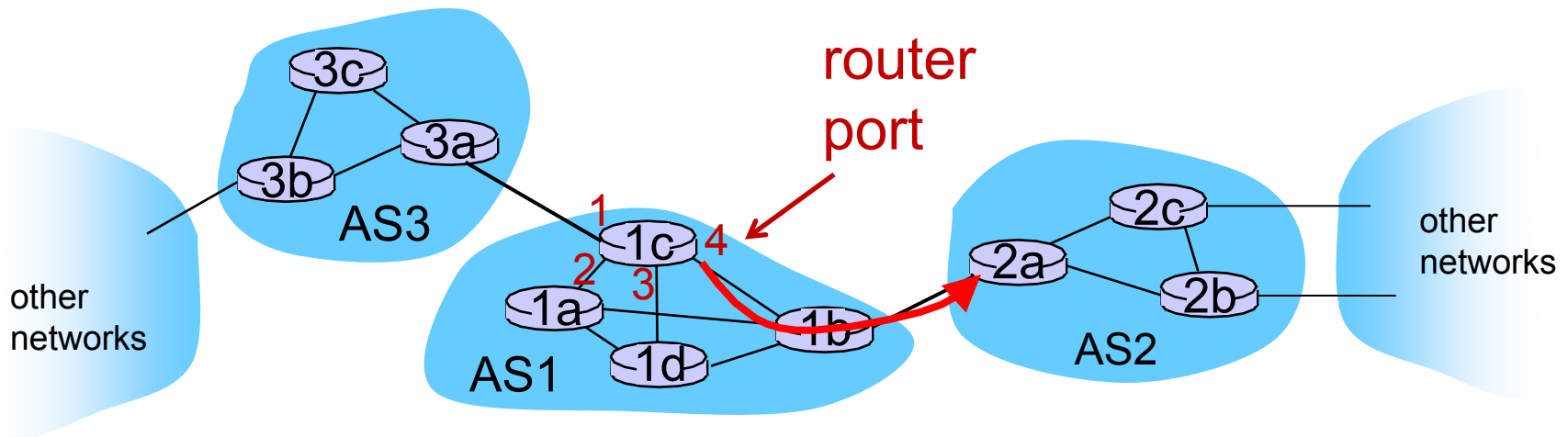
- ❖ Route from 1c to 111.99.86.55

ath



Router identifies port for route

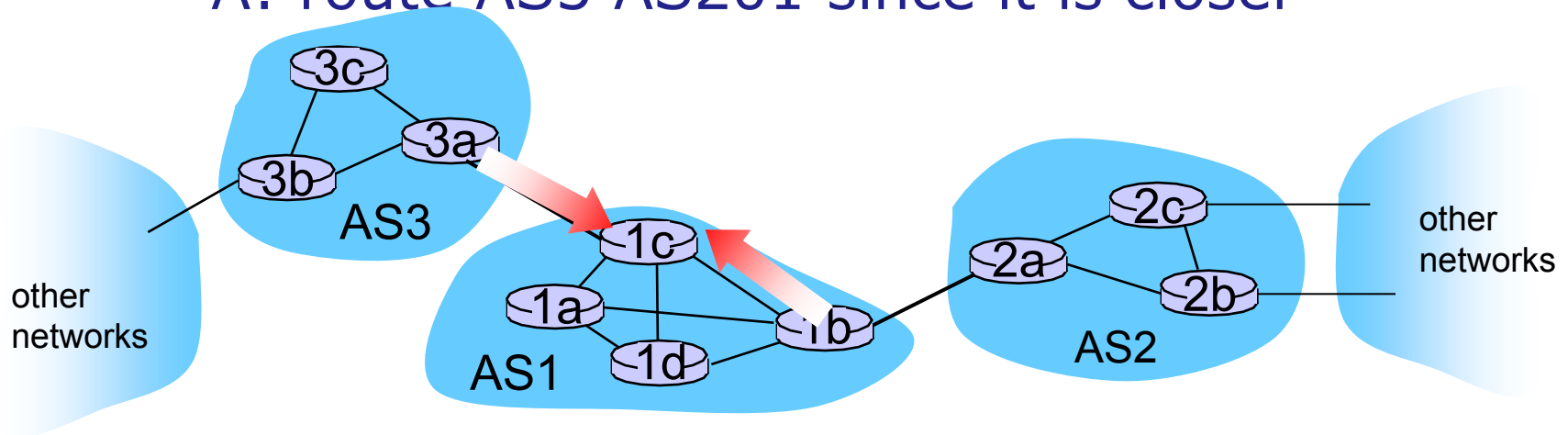
- ❖ Identifies port along the OSPF shortest path
- ❖ Adds prefix-
s forwarding
 - (1



Hot Potato Routing

- ❖ Suppose there two or more best inter-routes.
- ❖ Then choose the closest NEXT-HOP

- Q: AS17?
- A: route AS3 AS201 since it is closer

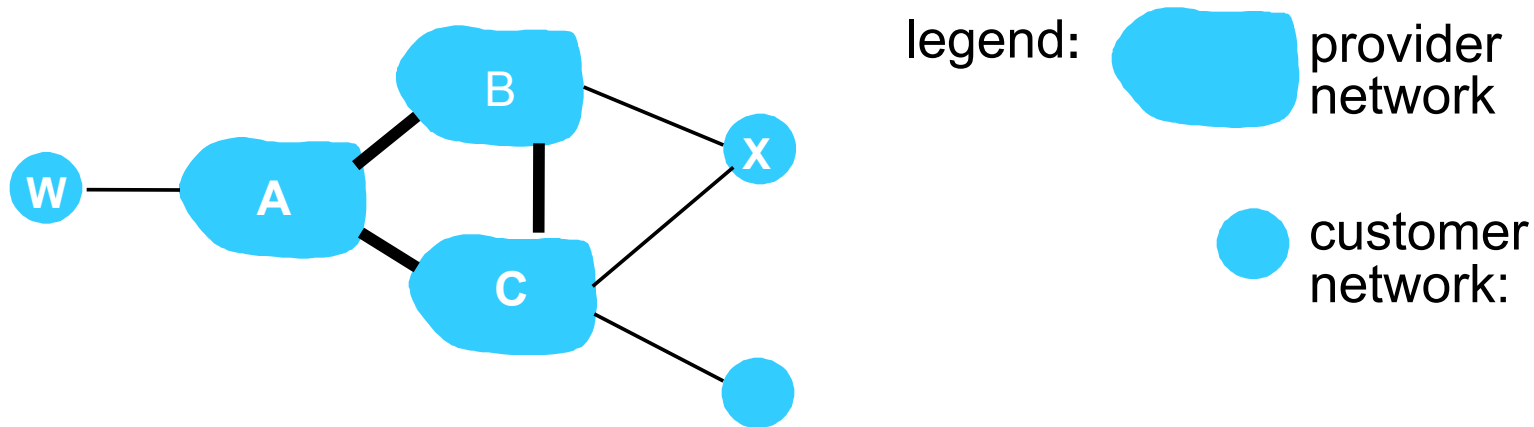


How does entry get in forwarding table?

Summary

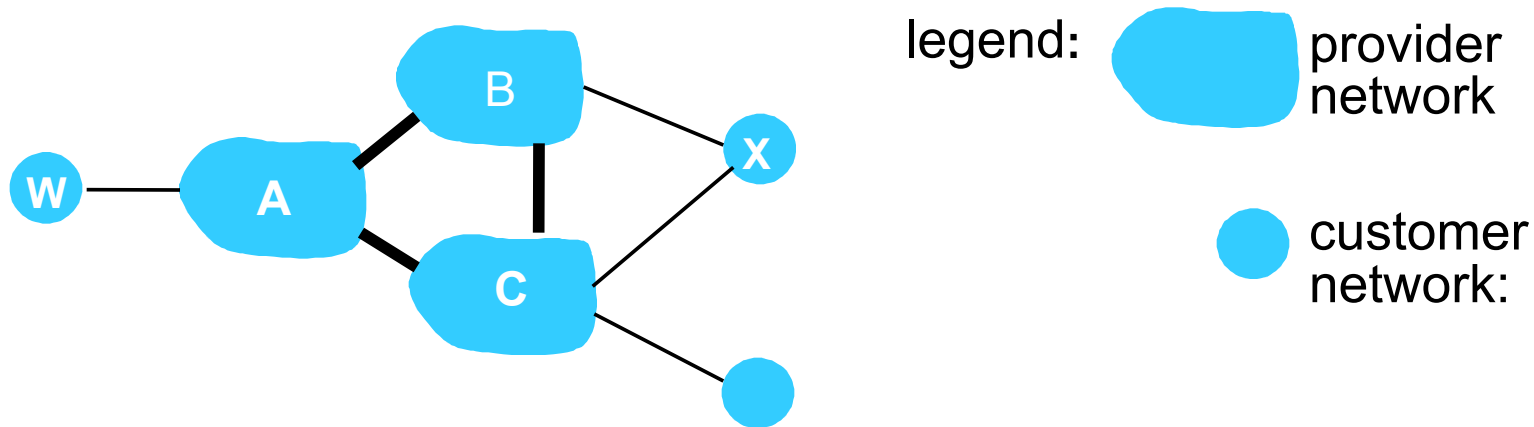
1. Router becomes prefix
2. Determine prefix
 - Use BGP route selection to find best inter-AS route
 - Use OSPF to find best intra-AS route leading to best inter-AS route
 - Router identifies router port for that best route
3. Enter prefix-port entry in forwarding table

BGP routing policy



- ❖ A,B,C are *provider networks*
- ❖ X,W,Y are customer (of provider networks)
- ❖ X is *dual-homed*: attached to two networks
 - X does not want to route from B via X to C
 - .. so X will not advertise to B a route to C

BGP routing policy (2)



- ❖ A adve
- ❖ B advertises path BAW to X
- ❖ Should B advertise path BAW to C?
 - No way! B gets no “revenue” for routing CBAW since neither W nor C are B’s customers
 - B wants to force C to route to w via A
 - B wants to route *only* to/from its customers!

Why different Intra-, Inter-AS routing ?

policy:

- ❖ inter-AS: admin wants control over how its traffic routed through its net.

scale:

- ❖ hierarchical routing saves table size, reduced update traffic

performance:

- ❖ intra-AS: can focus on performance
- ❖ inter-AS: policy may dominate over performance

Chapter 4: outline

4.1 introduction

4.2 virtual circuit and datagram netw

4.4 IP: I
Protocol

- datagram format
- IPv4 addressing
- ICMP
- IPv6

4.5 routing algorithms

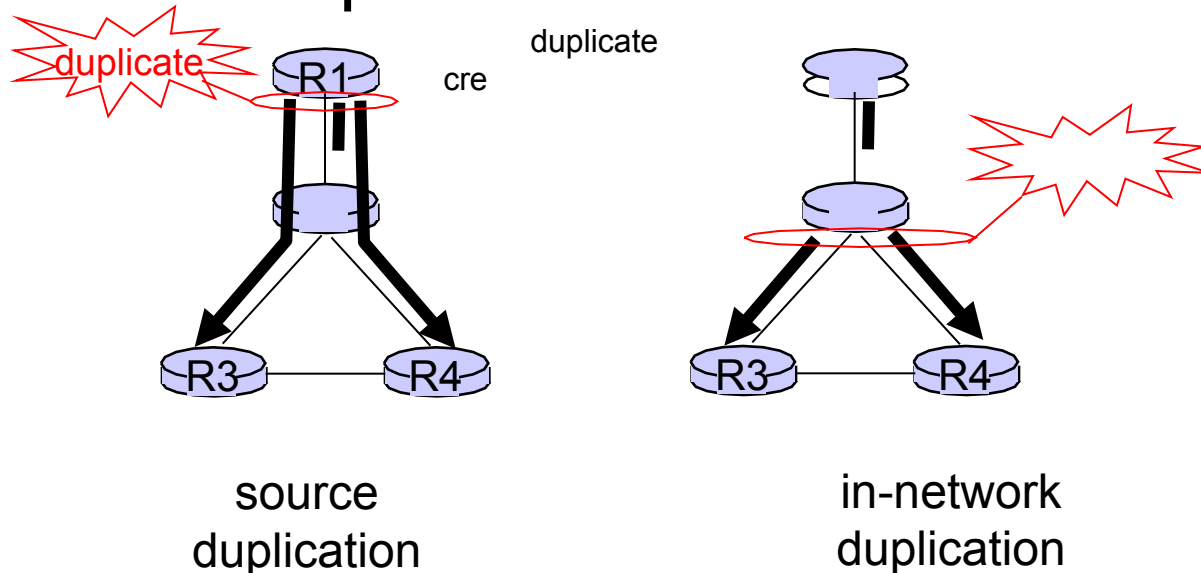
- link state
- distance vector
archical routing

- OSPF
- BGP

4.7 broadcast and
multicast routing

Broadcast routing

- ❖ deliver packets from source to all other nodes
- ❖ source duplication is inefficient:



- ❖ source duplication: how does source determine recipient addresses?

In-network duplication

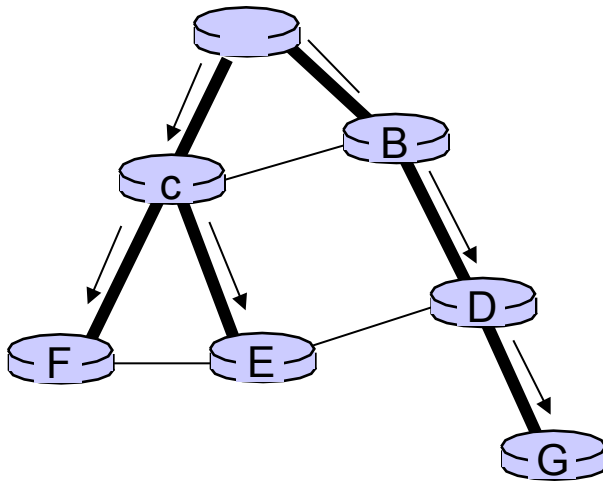
- ❖ *flooding*: when node receives broadcast packet, sends copy to all neighbors
 - problems: cy t storm

same

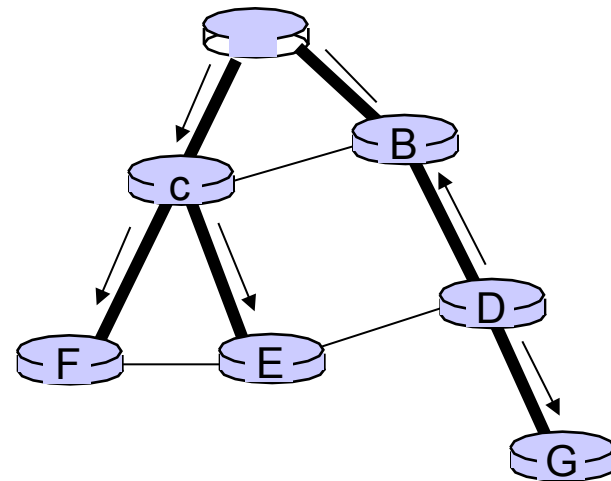
- node keeps track of packet ids already broadcasted
 - or reverse path forwarding (RPF): only forward packet if it arrived on shortest path between node and source
- ❖ *spanning tree*:
 - no redundant packets received by any node

Spanning tree

- ❖ first construct a spanning tree
- ❖ nodes then forward/make copies only along spanni



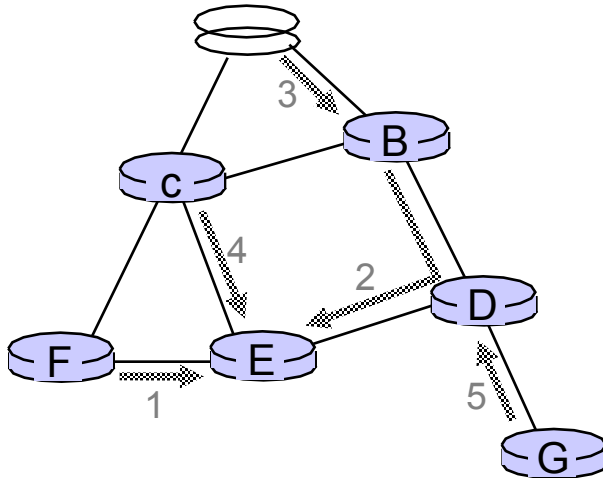
(a) broadcast initiated at A



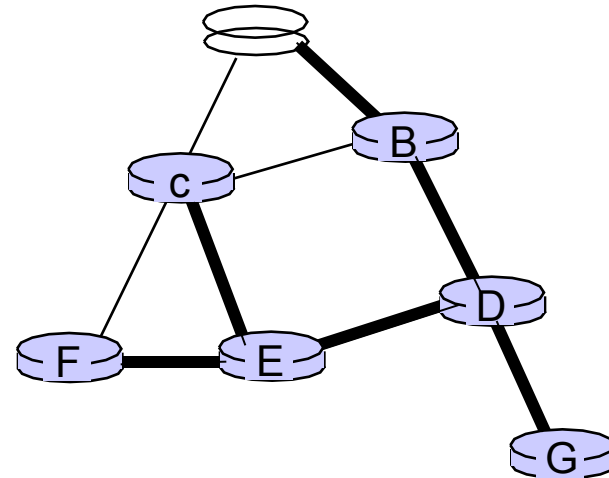
(b) broadcast initiated at D

Spanning tree: creation

- ❖ center node
 - ❖ each node sends unicast join message to center node
 - message for
- rives at a node



(a) stepwise construction of spanning tree (center: E)



(b) constructed spanning tree

Multicast routing: problem statement

goal: find a tree (or trees) connecting routers having local mcast group members

❖ *tree:* not all paths between routers used

❖ *shared-tree:* sa

❖

group members

legend



group member



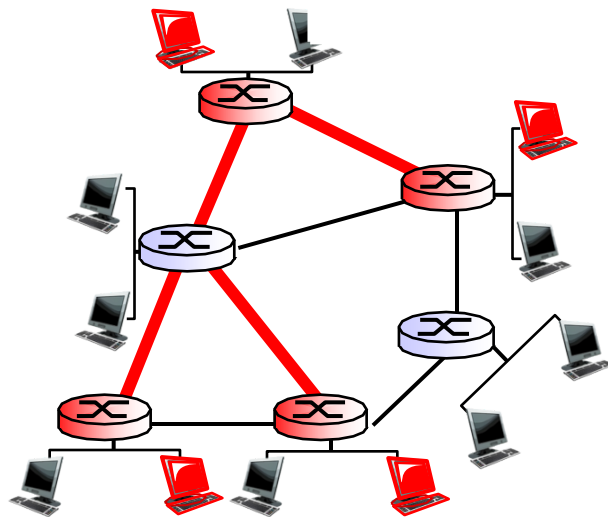
not group member



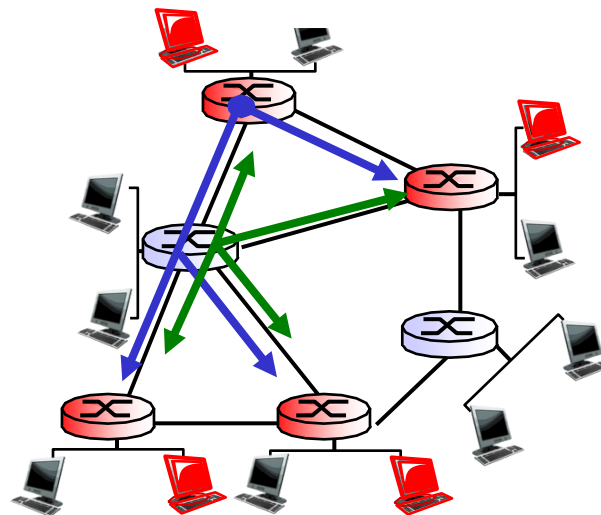
router with a group member



router without group member



shared tree



source-based trees

Approaches for building mcast trees

approaches:

❖ *source-based tree*: one tree per source

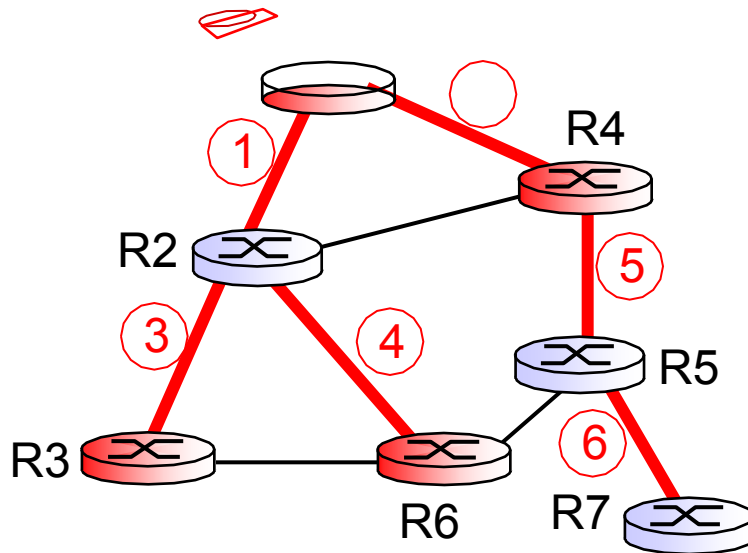
- shortest path trees
- reverse path

- mi
- center-based trees

...we first look at basic approaches, then specific protocols adopting these approaches

Shortest path tree

- ❖ mcast forwarding tree: tree of shortest path routes from source to all receivers
 - Dijkstra's alg



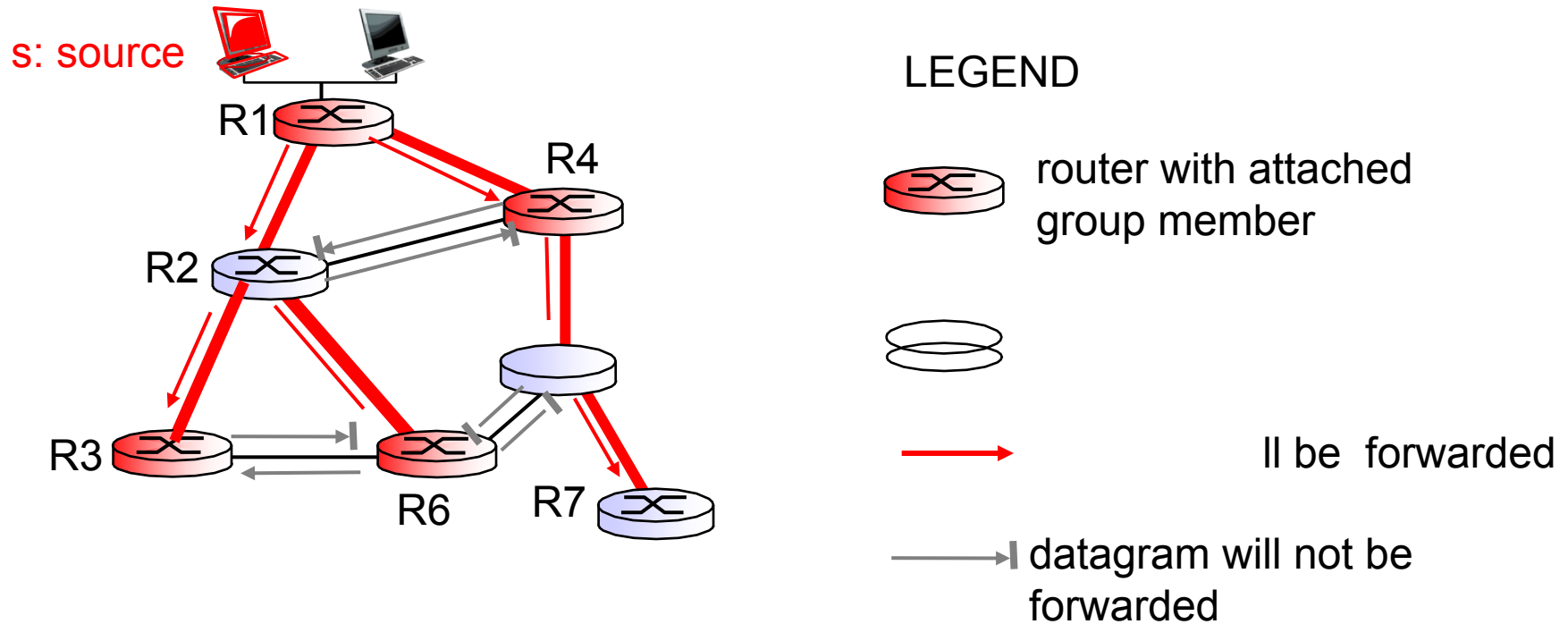
-  attached group member
-  router with no attached group member
-  link used for forwarding, i indicates order link added by algorithm

Reverse path forwarding

- ❖ rely on router's knowledge of unicast shortest path to destination

```
if (link on shortest path back to center)  
  then flood datagram onto all outgoing links  
  else ignore datagram
```

Reverse path forwarding: example

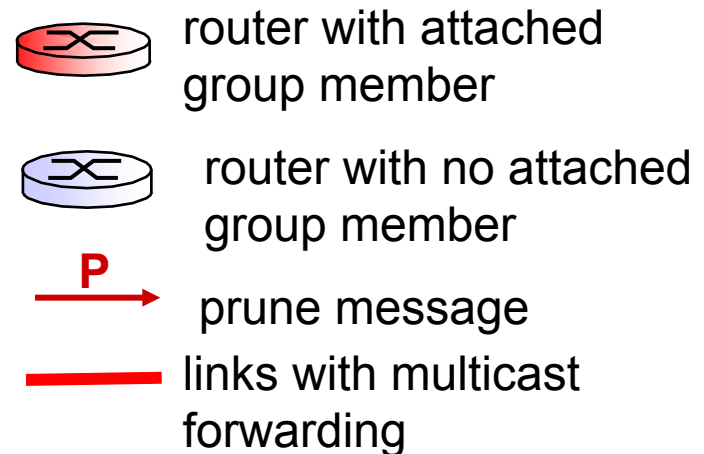
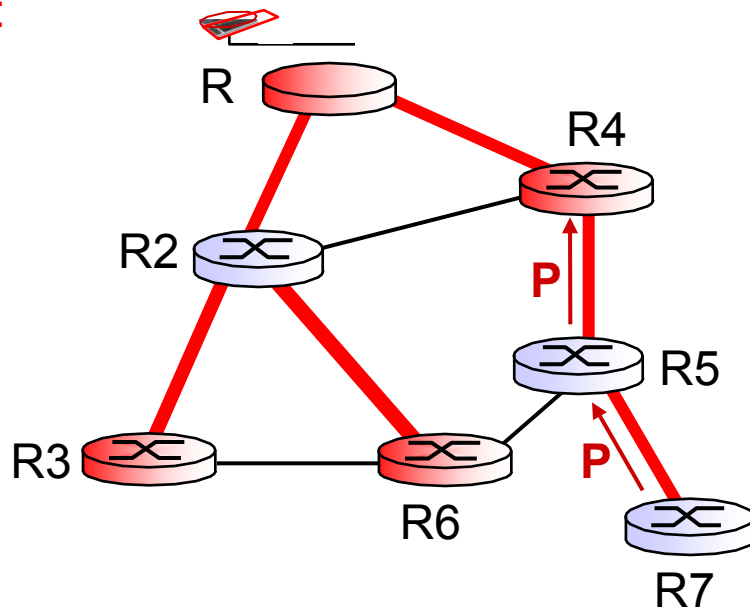


- ❖ result is a source-specific *reverse* SPT
 - may be a bad choice with asymmetric links

Reverse path forwarding: pruning

- ❖ forwarding tree contains subtrees with no mcast group members
 - no need to forward datagrams down subtree
 - “prune” msgs sent upstream by router with no downstreamers

S:



Shared-tree: steiner tree

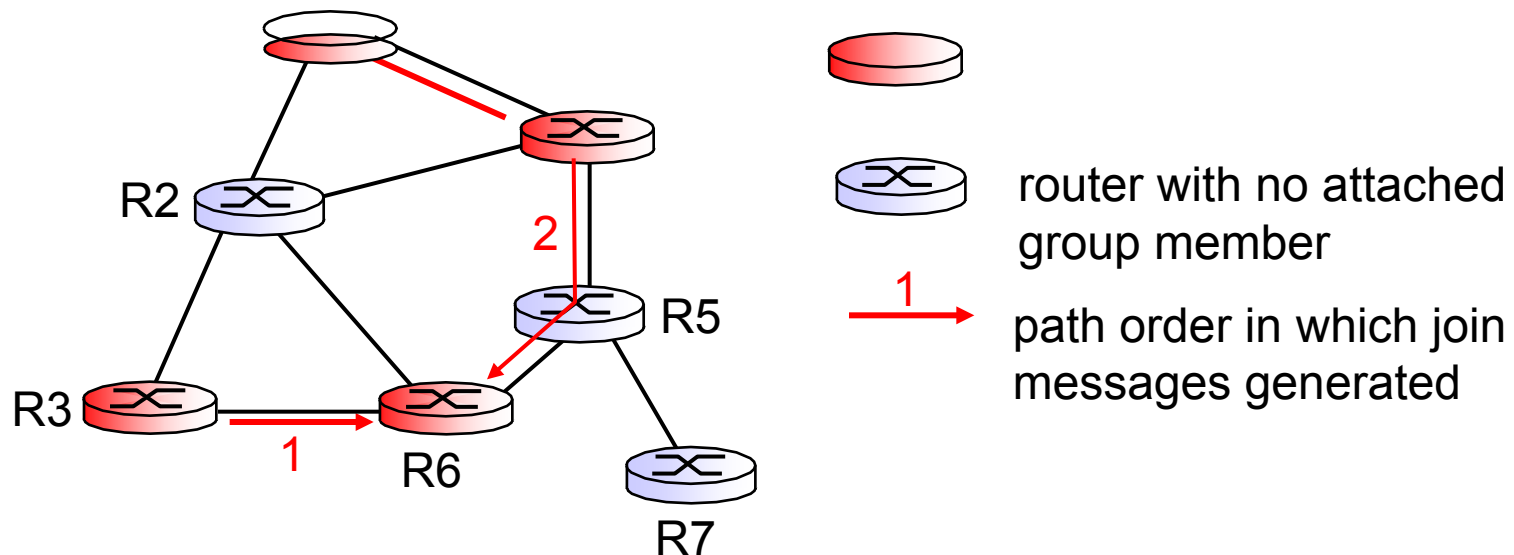
- ❖ *steiner tree*: minimum cost tree connecting all routers with attached group memb
- ❖
- ❖ not used in practice:
 - computational complexity
 - information about entire network needed
 - monolithic: rerun whenever a router needs to join/leave

Center-based trees

- ❖ single delivery tree shared by all
- ❖ one router identified as "*center*" of tree
- ❖ to join:
 - *joi*
routers and forwarded towards center
 - *join-msg* either hits existing tree branch for this center, or arrives at center
 - path taken by *join-msg* becomes new branch of tree for this router

Center-based trees: example

suppose R6 chosen as center:



Internet Multicasting Routing: ~~DVMRP~~

- ❖ **DVMRP**: distance vector multicast routing protocol, RFC1075
- ❖ *flood and prune* ath

tab

DVMRP routers

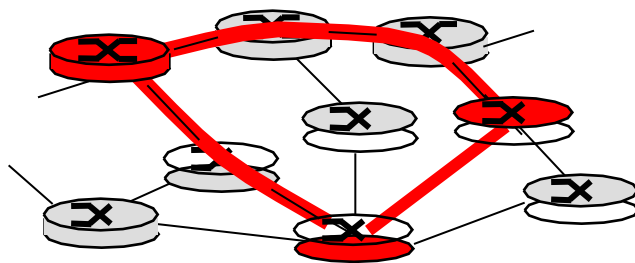
- no assumptions about underlying unicast
- initial datagram to mcast group flooded everywhere via RPF
- routers not wanting group: send upstream prune msgs

DVMRP: continued...

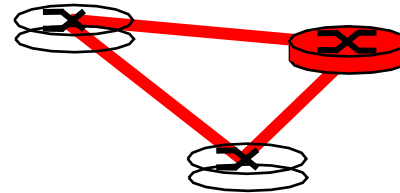
- ❖ *soft state*: DVMRP router periodically (1 min.) “forgets” branches are pruned:
 - mcast data again flows down unpruned branch
- ❖ route
 - following IGMP join at leaf
- ❖ odds and ends
 - commonly implemented in commercial router

Tunneling

Q: how to connect “islands” of multicast routers in a “sea” of unicast routers?



physical topology



logical topology

- ❖ mcast datagram encapsulated inside “normal” (non-multicast-addressed) datagram
- ❖ normal IP datagram sent thru “tunnel” via regular IP unicast to receiving mcast router (recall IPv6 inside IPv4 tunneling)
- ❖ receiving mcast router unencapsulates to get mcast datagram

PIM: Protocol Independent Multicast

- ❖ not dependent on any specific underlying unicast routing algorithm (works with all)
- ❖ group members densely packed, in “close” proximity.
- ❖ bandwidth more plentiful
- ❖ group members small wrt # interconnected networks
- ❖ group members “widely dispersed”
- ❖ bandwidth not plentiful

Consequences of sparse-dense dichotomy:

dense

- ❖ group membership by routers *assume* routers explicitl

- ❖ constr tree (e.g., RPF)
- ❖ bandwidth and non-group-router processing *profligate*

sparse:

- ❖ no membership until s explicitly join
- ❖ bandwidth and non-group-router processing *conservative*

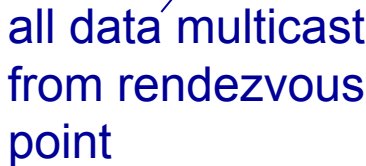
PIM- dense mode

flood-and-prune RPF: similar to DVMRP but...

- ❖ underlying unicast routing algorithm provides
- ❖ less downstream flood than DVMRP
reduces reliance on underlying routing algorithm
- ❖ has protocol mechanism for router to detect if it is a leaf-node router

~~mode~~

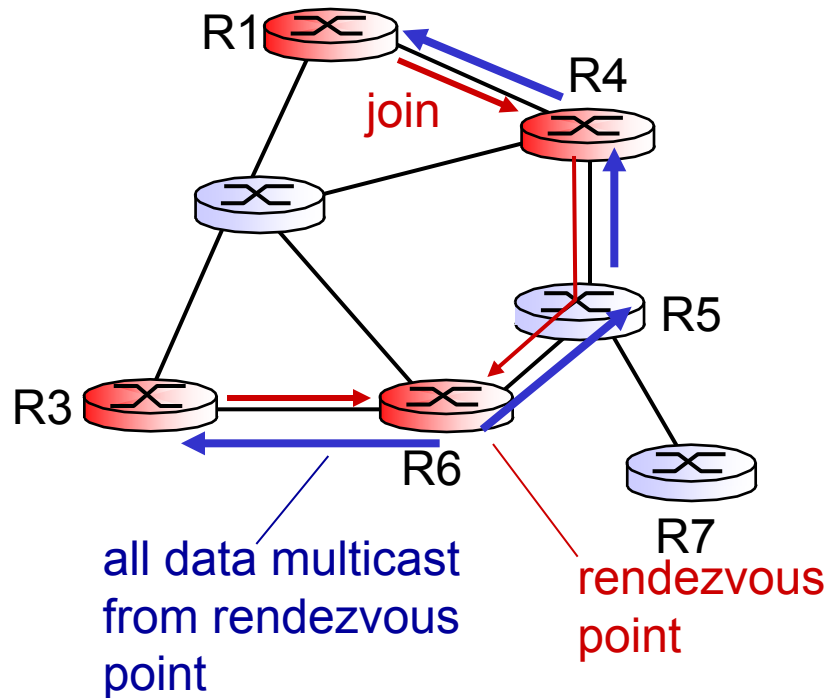
-



~~mode~~

sender(s):

- ❖ unicast data to RP, which
- ❖ RP c
mcast tree
upstream to
source
- ❖ RP can send *stop*
msg if no
attached
receivers



Chapter 4: *done!*

4.1 introduction

4.2 virtual circuit and datagram networks

4.3 what's inside

4.5 routing algorithms

- link state, distance vector, hierarchical routing

Proto

4.7 broadcast and

multicast routing

- datagram format, understanding principles behind network layer services, ICMP, IPv6
- ❖ network layer service models, forwarding versus routing how a router works, routing (path selection), broadcast, multicast
- ❖ instantiation, implementation in the Internet